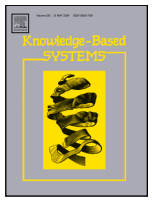
Knowledge-Based Systems 330 (2025) 114605



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





Multi-scale feature fusion network with temporal dynamic graphs for small-sample FW-UAV fault diagnosis

Guanjun Li [a], Haoyu Gui [a], Jianguang Lu [a,*], Xianghong Tang [a], Xiaoyu Gao [b]

[a] State Key Laboratory of Public Big Data, Guizhou University, 550025, Guiyang, China
[b] Ming Li College of Henan University of Economics and Law, 450046, Zhengzhou, China

ARTICLE INFO

Keywords:
FW-UAV
Fault diagnosis
Small sample
Spatiotemporal feature fusion
Temporal dynamic graph

ABSTRACT

With the extensive application of fixed-wing unmanned aerial vehicles, accurate fault diagnosis becomes crucial for flight safety and system reliability. Traditional fault diagnosis methods often require large datasets that are difficult to obtain in practice. To address this, we start with the spatio-temporal correlation characteristics and multi-dimensional heterogeneity of UAV flight data, and propose a multi-scale feature fusion with temporal dynamic graph network (MFFTD) that enables efficient fault diagnosis using limited UAV flight data. In the spatial dimension, a multi-scale residual convolutional design captures feature representations at various levels. Furthermore, the global temporal dynamic graph models the topological dependencies between the feature representations. In addition, we introduce long short-term memory networks to capture long-term dependencies in the temporal dimension. For cross-domain joint learning in both the temporal and spatial dimensions, we propose a multi-head feature fusion module based on mutual information to address the issue of heterogeneity imbalance between feature representations. Experiments on four public datasets demonstrated that MFFTD improves the detection accuracy by eight percentage points compared to the latest models under the 90 small-sample settings of multi-class tasks and significantly enhances the generalization capability, offering superior decision support for UAV fault diagnosis. The code and data will be available at https://github.com/17992/MFFTD.

1. Introduction

With the rapid development of unmanned aerial vehicles (UAV), they are increasingly used in a wide range of fields, including military reconnaissance [1], industrial surveillance [2], agricultural spraying [3], and logistics and transport [4]. The wide application of UAVs has significantly improved the work efficiency of these industries; however, it has also put forward high requirements for the safety of UAVs. Since UAVs performing tasks in complex environments [5] can be affected by the external environment, hardware failures, and software defects, continuous and stable flight operation are crucial. The UAV flight process generates a large amount of highly spatio-temporally correlated and multi-dimensional heterogeneous data, such as flight speed, altitude, attitude, temperature, and battery power, which can provide strong support for system health. Therefore, the utilization of flight data for fault diagnosis has become a key research direction for UAV applications. Currently, techniques used for UAV fault diagnosis can be divided into two categories: model- and data-driven fault diagnosis methods.

Model-based approaches employ mathematical and physical models to analyze flight data, thereby identifying anomalies and faults. For example, Telford and Galloway [6] proposed a UAV electrical power system fault classification and diagnostic system based on hidden Markov models to assist and improve UAV health management and control. However, this method is only applicable to the binary classification of fault diagnosis tasks. In [7], a quadrotor was modeled as a linear parameter varying (LPV) system, and designed an LPV observer for fault detection; however, the LPV model was applied only to nonlinear systems within a narrow scope and was insufficient to capture complex dynamic characteristics. Another study [8] presented a mathematical model for a quadrotor UAV and proposed a robust nonlinear controller that combined the sliding-mode control and backstepping control techniques, which achieved good diagnostic results in simulation experiments. However, this method relies heavily on an accurate mathematical model of the UAV and has a poor generalization ability. In [9], actuator fault diagnosis for multirotor UAVs was proposed based on the cascade of a nonlinear Thau observer and a linearized Kalman filter. However, this method depends on the accurate linearization of the system, and complex nonlinearities impair the diagnostic performance [10] proposed an extended Kalman filter (EKF)-based fault observer for hexacopter UAV fault diagnosis and isolation and validated the effectiveness of the method on simulation data. However, the EKF is sensitive to noise and modeling errors and is prone to stability reduction by error ampli-

* Corresponding author.
E-mail address: jglu@gzu.edu.cn (J. Lu).

https://doi.org/10.1016/j.knosys.2025.114605
Received 2 June 2025; Received in revised form 19 September 2025; Accepted 1 October 2025
Available online 9 October 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

fication in practice [11] proposed an actuator fault detection and diagnosis method based on EKF and multi-model adaptive estimation. However, this method incurs high development and maintenance costs and has limited versatility. Model-based approaches require accurate system modeling, have high development and maintenance costs, and vary from one UAV to another; thus, these approaches face many challenges in practical applications.

Data-driven methods, which are adaptable to learning features from data for fault diagnosis, have gained considerable attention in recent years. In [12], real flight data on a small fixed-wing unmanned aerial vehicle (FW-UAV) with an open-source Paparazzi autopilot system were presented, and support vector machine (SVM) algorithm was used for fault diagnosis. However, it encountered an overfitting problem. In [13] signal preprocessing and feature extraction on sound data obtained from electric motors was performed using a decision tree, SVM, and k-nearest neighbors for fault diagnosis. However, it relies on manual feature engineering, making it difficult to capture the complex patterns and multiscale characteristics of fault signals, thereby limiting the diagnostic accuracy. Reference [14] used graph attention networks (GAT) for fault diagnosis by treating flight data as graph structures. However, it has a low efficiency in high-dimensional graph feature learning and requires a large amount of labeled data for network training. In [15], a novel UAV fault diagnosis approach based on self-supervised contrast learning and spatial-temporal information of multivariate flight data was proposed; however, it requires large amounts of data and is sensitive to hyperparameters. The above methods relied heavily on the quality and quantity of the data and had limited performance on high-dimensional time series data. To address the scarcity of UAV failure data [16] extracted temporal and spatial features from flight data by employing tandem convolutional neural networks (CNNs) and long short-term memory networks (LSTM), aiming to solve the problem of troubleshooting FW-UAVs with few-shot learning. In [17], the heterogeneous deep multitask learning (HDMTL) framework with adaptive sharing and knowledge complementation was used to explore fault diagnosis under small samples. Research [18] focused on small sample multi-task fault diagnosis, enhancing model performance through multi-task joint training and proposing a novel scheme for reliable diagnosis of complex UAV systems. These models consider the problem of scarce and expensive UAV failure data; however, they are too coarse to acquire the temporal and spatial characteristics of flight data, and there is considerable scope for enhancement.

To address the above challenges, we propose fault diagnosis model **M**ulti-Scale **F**eature **F**usion with **T**emporal **D**ynamic graph network (MFFTD) for FW-UAV fault diagnosis. At the spatial feature extraction level, we designed a multi-scale residual convolutional module for capturing spatial information, and to capture spatial topological information, we introduced a global temporal dynamic graph to enhance the above convolution process. For timing-feature capture, we used LSTM to model the long-term dependence between timings. For feature fusion, we designed a multi-head mutual information self-attentive feature fusion module to address the data imbalance problem between spatial and temporal features, along with an advanced feature abstraction fusion process. Overall, our main contributions can be summarized as follows:

1. To the best of our knowledge, this is the first study to introduce global temporal dynamic graphs into small-sample drone fault diagnosis, capturing long-term spatial topological dependencies among anomalous data. This approach addresses the limitations of traditional methods in capturing the spatio-temporal characteristics of flight data.
2. To address data scarcity and imbalance, we design a multi-head mutual information self-attention feature fusion method. Through the combination of multi-head self-attention and mutual information constraints, the accuracy and robustness of anomaly detection are enhanced.
3. In designing the spatio-temporal feature extraction backbone, we incorporate a multi-scale residual convolutional method. By deeply integrating multi-scale feature extraction with residual learning, we achieve comprehensive capture and efficient representation of complex spatial features in flight data.
4. We conduct extensive evaluations on four publicly available real-world fixed-wing drone flight datasets. The experimental results demonstrate that MFFTD achieves performance across multiple metrics in fault diagnosis accuracy.

The remainder of this paper is organized as follows. Section 2 briefly introduces the existing related research. Section 3 presents the MFFTD framework and implementation process. In Section 4, we describe the dataset, data processing, experimental analysis, ablation experiments, and visual analysis. Section 5 summarizes this study.

2. Related works

In this section, we briefly review UAV fault diagnosis, multiscale residual convolutional networks, relevant advances in temporal dynamic graphs, and attention mechanism methods for UAV fault diagnosis.

2.1. UAV Fault diagnosis

Fault diagnosis of UAV flight data is crucial for ensuring flight safety with the aim of accurately identifying potential faults by analyzing multi-dimensional time-series data [19] (such as those from sensors, control commands, and environmental parameters) during the flight process. Current research mainly relies on deep-learning models for feature extraction and fault classification [20], and these methods usually achieve relatively high accuracy when there is a large amount of labeled data. However, in practical applications, owing to the complex operating environments of UAVs and limitations in data collection, an insufficient number of samples often restricts the performance and generalization ability of the models. In addition, UAV flight data exhibit significant spatio-temporal dependence characteristics, including local dynamic changes and global long-term correlations. Nevertheless, most existing methods adopt a single feature extraction mechanism [21], making it difficult to model multi-scale features simultaneously, which leads to insufficient representation of local details or a lack of ability to capture global trends. Few-shot learning has the advantage of efficiently mining deep features and reducing the cost of data collection for UAV flight failures. It is a research direction with both academic value and practical significance and can effectively solve problems such as data scarcity, label imbalance, and insufficient generalization ability.

2.2. Multi-scale residual convolutional network

Since the advent of CNN [22], significant progress has been made in their application to drone fault diagnosis research. We adopt a tabular review approach [23] for organization and summarization as shown in Table 1.

However, traditional CNNs have limitations in feature extraction such as a fixed receptive field, poor adaptability to scale variations, and the vanishing gradient problem, which hinder their ability to capture complex patterns. In the context of fault diagnosis for UAV flight data, CNNs struggle to balance global and local feature extraction, thereby constraining diagnostic performance. We have previously introduced a multi-scale residual convolutional network (MRCN) [28] in the field of UAV fault diagnosis for the first time. By integrating convolutional kernels of varying sizes, MRCN effectively expands the receptive field, enabling the simultaneous capture of global trends and local details within the data. This capability is particularly advantageous for handling scale-variant characteristics inherent in fault signals. Furthermore, the incorporation of residual connections [29] mitigates the vanishing gradient

Table 1
The application of CNN in UAV fault diagnosis.

Research	Main Contributions	Unique Aspects
SCNN[24]	Proposes a CNN-based UAV sensor fault diagnosis method using short time fourier transform (STFT) for signal conversion	Innovatively integrates SFTT and CNN, transforming residual signals into time-frequency maps for CNN to extract sensor fault information
CNN-KF[25]	Proposes a novel fault diagnosis approach for micro-electromechanical systems (MEMS) inertial sensors that recognize the fault patterns of MEMS inertial sensors in an end-to-end manner	Introduces an improved adaptive learning rate optimization method which accelerates the loss convergence by using the Kalman filter to train the network efficiently with a small dataset
1D-CNN[26]	Proposes a method integrating vibration signal interval sampling reconstruction with 1D-CNN, and improves the recognition rate for different degrees of faults	combines vibration signal interval sampling with 1D-CNN for enhanced feature extraction, enabling sample sets to capture global operational data
AdaBN-1DCNN[27]	Propose a one-dimensional convolutional neural network model based on the adaptive batch normalization (AdaBN) algorithm to improve the CNN model	Innovatively uses AdaBN to adjust batch normalization layer parameters during testing for better domain adaptability under variable conditions

Table 2
The application of attention mechanism in UAV fault diagnosis.

Research	Main Contributions	Unique Aspects
ASN[36]	Proposes an adaptive Siamese network with a channel attention mechanism to adaptively weight channel features, and apply it to UAV fault diagnosis under limited data	Innovatively uses the channel attention mechanism to adaptively enhance feature weights based on channel importance, and reduce multichannel data coupling
MISAL[20]	Proposes a sensor fault diagnosis method combining self-attention mechanism for similarity space mapping of the value matrix, and introducing LSTM to learn spatial-temporal data correlations	Integrates the self-attention mechanism to achieve similarity space mapping, overcoming the limitation of single similarity calculation in conventional attention-based fault diagnosis methods
ST-GATrans[37]	Proposes enhanced graph attention network with graph multiheaded self-attention and develops a transformer encoder combining multiheaded self-attention and convolutional token fusion for temporal feature mining	Dual-attention for flight spatio-temporal features: enhanced graph attention with multiheaded self-attention for spatial, transformer with convolutional fusion for temporal
CSGTOF[38]	Proposes a multi-task CSGTOF framework integrating generic feature sharing network and attention mechanism for FW-UAV small-sample fault diagnosis, meeting real-time needs	Innovatively combines attention mechanism with generic-task feature collaborative sharing to enable unique cross-task knowledge complementation in small-sample FW-UAV fault diagnosis
MST-BGCT [39]	Proposes MST-BGCT for UAV swarm distributed fault diagnosis and integrates multiple functional modules to enable high-quality and reliable fault diagnosis	Innovatively combines graph attention network with convolutional transformer, and introduces Bayesian framework to address uncertainty, tackling fault features' high nonlinearity and spatio-temporal coupling

problem, thereby improving training efficiency and feature representation in deep architectures. These advantages underscore the potential of MRCN as a highly effective approach for UAV fault diagnosis, demonstrating exceptional applicability in this domain.

2.3. Global temporal dynamic graphs

Graph neural networks (GNN) have been demonstrated to effectively capture complex dependencies in temporal data, showcasing powerful capabilities in feature extraction and relationship modeling. In recent years, GNNs have attracted increasing attention in the field of flight fault data diagnostics. CLGNN [30] leverages GNNs to model sensor signals represented in a graph structure. A temporal dynamic graph (GTDG) is a graph network design capable of effectively addressing the variability of temporal data and dynamic nature of graph structures. GTDG has achieved significant success in multidimensional temporal data analysis [31]. TD-Graph-LSTM [32] was proposed to address action-driven, weakly supervised object detection by propagating the temporal context over a per-frame dynamic graph structure, which can alleviate the problem of missing labels per frame. In the field of multivariate time-series classification, TodyNet [33] was designed as an end-to-end temporal graph neural network to capture dynamic time-series features adequately through dynamic graphs. However, during temporal dynamic graph fusion, TodyNet suffers from error accumulation and cannot handle missing or unequal-length data. CATodyNet [34] was proposed to address these issues. However, the aforementioned methods focused on designing temporal dynamic graphs from a local perspective of long-term dependencies, making it challenging to directly model flight fault data with limited samples. Consequently, the application of temporal dynamic graphs from a global perspective in the domain of small-sample flight fault diagnostics requires further investigation.

2.4. Attention-based methods for UAV fault diagnosis

Owing to the successful application of attention-based transformer architectures in natural language processing [35], an increasing number of attention mechanisms are being incorporated into UAV fault diagnosis. The most recent results are presented in Table 2.

However, these methods overlook the imbalance between spatio-temporal features and the accumulation of information errors during fusion. In this study, we proposed an MHMIFF module. It employs a multi-head mechanism to compute spatiotemporal mutual information independently, and encodes multidimensional feature correlations to retain discriminative information while eliminating redundancy. During the fusion, a linear layer with normalization and dropout prevents dimensional explosions and suppresses ineffective features, thereby enabling efficient and compact feature fusion.

3. Methodology

In this section, we first explain the problem formulation of fault diagnosis, and then provide a detailed description of the MFFTD model framework, including the GTDG, MRCN, LSTM, and **M**ulti-**H**ead **M**utual **I**nformation **F**eature **F**usion Network (MHMIFF), as shown in Fig 1. Finally, we perform a detailed time complexity analysis of the model based on the pseudocode of MFFTD.

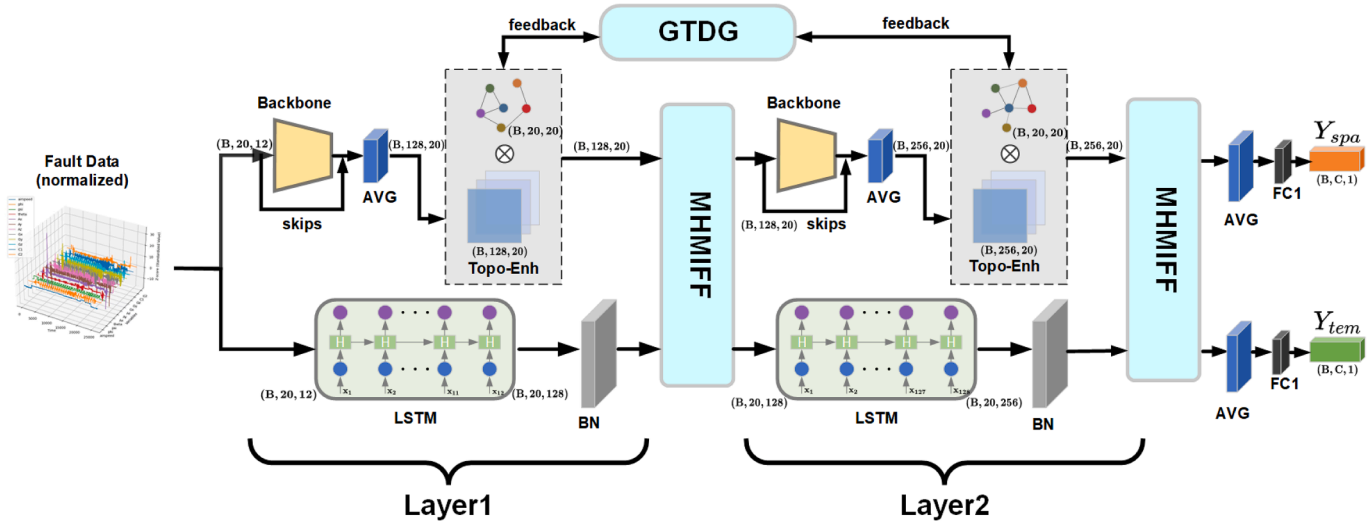


Fig. 1. Structure diagram of two-layer MFFTD. Spatial feature capture and temporal dependency capture are performed in parallel. A possible regularized fault data flow example, with dimensions (B, T, F), represents batch size B, the number of channels (20 in this case), and feature length F (12 in this case). Specifically, during the spatial feature capture process, the features extracted by the backbone are further averaged through pooling (AVG) and then combined with the GTDG provided for feedback, further enhancing the topology. In the temporal dependency capture process, the LSTM outputs hidden layer features, which are subsequently used for batch normalization (BN). These features are then further processed by the MHMIFF to extract high-level features. The above components are stacked to form the two-layer MFFTD, and finally, AVG and fully connected layers are used to obtain spatial prediction Y_{spa} and temporal prediction Y_{tem}. C is the number of fault categories and the ⊗ denotes concatenation.

3.1. Problem formulations

FW-UAV flight data record the vehicle's flight status with strong spatiotemporal correlations and multidimensional heterogeneity. Assume that is a dataset available for training $F = \{(X_i, Y_i)\}_{i=1}^{S}$, where X_i and Y_i are the samples and their labels, respectively, and S is the number of samples. The sample $X = \{x_t\}_1^T$ is a sequence of lengths T, where each time step x_t is an F-dimensional vector. Formally, $x_t = [x_t^1, x_t^2, \ldots, x_t^F]$, where x_t^F is the value of the Fth feature at time t. Specifically, the vector form of the sample is $X \in \mathbb{R}^{T\times F}$. The fault sample $X_i \neq f(z)$, where $f(z)$ represents the normal operation mode of the aircraft and z denotes the operational input of the aircraft. In the fault diagnosis process, the spatial features In_{spa} and temporal features In_{tem} are captured by the MRCN, GTDG, and LSTM networks, respectively, and then undergo feature fusion through the MHMIFF.

3.2. Backbone

To capture features in both local and global contexts from fault data, inspired by the design of ResNet [40], we propose a lightweight and efficient multi-scale residual convolutional skeleton network based on multi-kernel feature extraction. Specifically, the network uses multiple convolutional kernels to capture fault features, which significantly enriches fault expression patterns in different semantic contexts. In addition, we employed linear mapping to fuse information across channels from the original fault data, achieve a lightweight design, and enable more complex feature relationship learning while maintaining or enhancing the model's expressive power. Given the input $X \in \mathbb{R}^{T\times F}$, the convolution with size kl is calculated as follows:

$$Y_o(t) = \sum_{m=0}^{T-1}\sum_{i=0}^{kl-1} X_m(t-i)\cdot K_{om,i} + b_o \tag{1}$$

where $Y_o(t)$ represents the value at time step t of the output Y_o, $X_m(t-i)$ is the value at time step $t-i$ of the mth input channel in X, $K_{om,i}$ is the weight at position i for the mth input channel and oth output channel in the convolutional kernel, and b_o is the bias term corresponding to output channel o. Finally, we use a padding of $\frac{kl-1}{2}$ to ensure consistency between the input and output dimensions.

In terms of designing the nonlinear expressive capability of the network, the ReLU activation function is commonly used to improve computational efficiency and accelerate training. We used a Leaky ReLU activation function to avoid the gradient vanishing problem on the negative half axis of ReLU. For a given input Z_o, the Leaky ReLU function is calculated as follows:

$$A_o(t) = \begin{cases} Z_o(t) & \text{if } Z_o(t) > 0 \\ \alpha \cdot Z_o(t) & \text{if } Z_o(t) \le 0 \end{cases} \tag{2}$$

where $A_o(t)$ is the output of the activation function, $\alpha = 0.01$ is the slope of the negative half-axis, and $Z_o(t)$ is the output $Y_o(t)$ after batch normalization.

To improve the model's robustness and mitigate issues such as vanishing or exploding gradients, batch normalization is applied to normalize the output of the convolutional layers, and dropout regularization is used to reduce model overfitting. Based on the above discussion, the backbone is shown in Fig 2.

In the MFFTD architecture, the backbone serves as the core for spatial feature extraction. By utilizing multi-scale convolution, it extracts fundamental fault features and provides high-quality feature inputs for GTDG and MHMIFF. As a crucial link between raw input and high-level feature processing, it significantly affects the overall fault diagnosis performance.

3.3. GTDG

As discussed, the existing work still has limitations in capturing spatial topological relationships. Influenced by CATodyNet [34], we enhanced the capture of spatial topological relationships by introducing GTDG into the backbone. Furthermore, we hypothesized that this spatial topological relationship has transitivity in the global context, meaning that as the deep neural network is constructed, this global relationship can be captured by the graph and sequentially passed to the next layer.

In contrast, we define the dynamic graph using an initialization approach, where the adjacency matrix is fully populated with ones, representing a complete graph structure at the beginning. In the context of few-shot learning for fault diagnosis, a dynamic graph does not

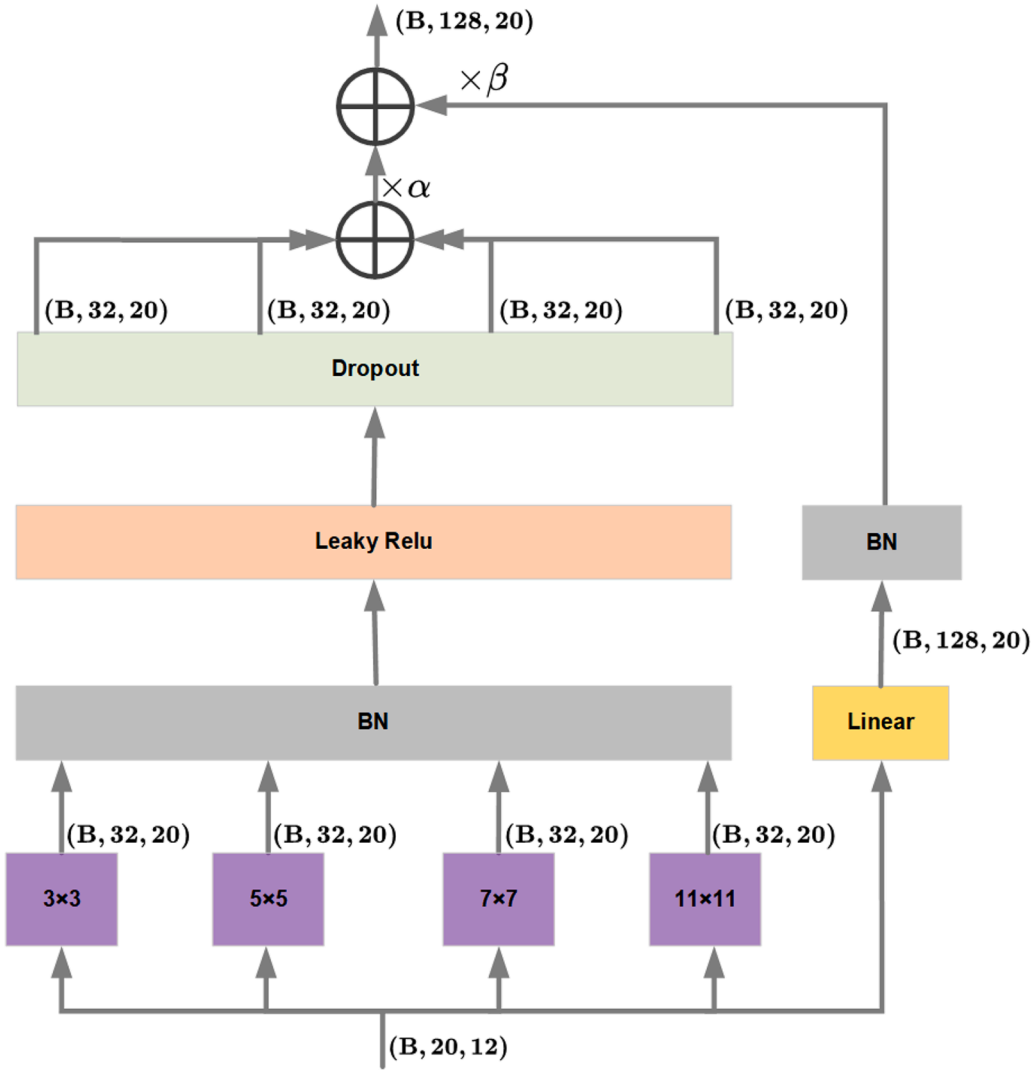


Fig. 2. The backbone of MFFTD. Four convolutional kernels of different sizes are used for average channel feature extraction. The linear layer implements residual connections through channel mapping. Learnable parameters α and β are used for model weight fitting. The $\oplus$ denotes tensor multiplication.

exhibit partition cutting properties. This preserves the integrity of the features captured by the backbone to some extent; however, feedback learning of the dynamic graph enables better learning of the topological spatial dependencies (consistent in the global context). Specifically, a GTDG can be represented by G as follows:

$$G = \{(V, E)_l\}_{l=1}^{L} \tag{3}$$

$$V = \{v_1, v_2, \ldots, v_n\} \tag{4}$$

$$E = \{e_{11}, e_{12}, \ldots, e_{1n}, \ldots e_{nn}\} \tag{5}$$

where L represents the number of layers in MFFTD, V is the set of n nodes, and E is the set of $n \times n$ relations (edges). On one hand, V is considered a static set that remains unchanged over time. On the other hand, the number of relations in E varies depending on layer l. During the initialization phase, we randomly initialize E as follows:

$$e_{ij}(l=1) = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1-p \end{cases} \tag{6}$$

where p is the probability of the existence of an edge (e.g., $p = .05$) and $e_{ij}(l = 1)$ represents whether an edge exists between nodes v_i and v_j at the initial layer $l = 1$.

Then, we construct the initialized E into a relational adjacency matrix adj_{init}:

$$adj_{init} = \begin{pmatrix} e_{11} & e_{12} & \cdots & e_{1n} \\ e_{21} & e_{22} & \cdots & e_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ e_{n1} & e_{n2} & \cdots & e_{nn} \end{pmatrix} \tag{7}$$

and set it as a learnable parameter to enable feedback learning during the backpropagation process, as follows:

$$A_{oe}^{l} = f_{topo-enh}^{l}\left(A_o^l, adj_{init}\right) \tag{8}$$

$$adj_l = f_{feedback}^{l}\left(A_{oe}^{l}\right) \tag{9}$$

where $f_{topo-enh}^{l}$ represents the topological enhancement, A_o^l is the output of the lth layer of the backbone, $f_{feedback}^{l}$ represents the feedback learning process using the enhanced output A_{oe}^{l}, and adj_l denotes the updated dynamic relationship representation after learning.

Specifically, the dynamic adjustment of the graph connectivity of GTDG is achieved through the coordination of the learnable adjacency matrix and gradient feedback mechanism. First, in the initialization phase, the adjacency matrix adj_{init} is defined as a learnable parameter initialized as an all-one matrix. During the forward propagation process, the backbone network outputs A_o^l and adj_{init} undergo topological

enhancement operations $f^{l}_{topo-enh}$ to generate the enhanced feature A^{l}_{oe}. Subsequently, the feedback function $f^{l}_{feedback}$ performs backpropagation learning on A^{l}_{oe}. During backpropagation, the model optimizes adj_{init} based on the loss function, strengthening important connections with large gradient magnitudes, and suppressing redundant connections with smaller gradient values.

In the MFFTD framework, the GTDG module not only precisely captures spatial features and models their topological dependencies but also works synergistically with the temporal feature module to comprehensively analyze the spatiotemporal characteristics of fault data, thereby enhancing diagnostic accuracy and robustness.

3.4. LSTM

A core issue in the field of fault diagnosis is the modelling of long-term temporal dependencies, which manifests as the influence of the current state at time step t on a future time point. This effect may be the result of long-term effects. In general, existing studies have tended to use reliable LSTM [41] networks to address this issue. The design of the LSTM's input, forget, and output gates allows the model to flexibly retain critical long-term memory, which enhances its ability to capture features along the temporal dimension to some extent. In this section, we do not discuss the specific internal implementation of LSTM in detail, as we reserve this part for reference in related literature. Instead, we briefly outline the role of LSTM in MFFTD from the perspective of data-flow processing. Specifically, given the input $X \in \mathbb{R}^{T\times F}$, LSTM performs the following updates:

First, the input gate In^{t}_{g} determines the extent to which the input at the current time step t can be passed to the cell state, as follows:

$$In^{t}_{g} = \sigma(W_i \cdot [H_{t-1}, X] + B_i) \tag{10}$$

where σ is the sigmoid activation function, W_i is the weight matrix of the input gate, and B_i is the bias term. H_{t-1} denotes the previously hidden state.

Next, the forget gate Fo^{t}_{g} controls the cell's forgetting threshold, determining how much of the historical information should be retained or discarded:

$$Fo^{t}_{g} = \sigma(W_f \cdot [H_{t-1}, X] + B_f) \tag{11}$$

where W_f is the weight matrix for the forget gate and B_f is the bias term.

After the above process, a portion of the candidate cell state will be updated, determining how much of the previous cell state will be fused with the current input for the update. Finally, the output gate of the LSTM generates a new output while updating the hidden state.

$$Ou^{t}_{g} = \sigma(W_o \cdot [H_{t-1}, X] + B_o) \tag{12}$$

$$H_t = Ou^{t}_{g} \cdot \tanh(CE_t) \tag{13}$$

where B_o is the bias term and W_o is the weight matrix for the output gate. The final hidden state is H_t, CE_t is the updated cell state, and tanh is the hyperbolic tangent activation function. MFFTD uses this hidden state to capture temporal dependencies and, together with the topologically enhanced input, further processes it for high-level feature abstraction.

In the MFFTD architecture, the LSTM module models temporal dependencies by extracting key sequential features from the fault data through a gating mechanism, enabling temporal prediction Y_{tem}. These features are then fused with topologically enhanced spatial inputs in MHMIFF for high-level feature abstraction. As such, LSTM not only captures temporal patterns, but also underpins the collaborative spatiotemporal processing essential for effective fault diagnosis.

3.5. MHMIFF

A notable feature of MFFTD is that its feature capture approach, which is based on both spatial and temporal dimensions, significantly enhances the ability to learn rich latent patterns. However, this parallel design lacks effective prior knowledge and strategies for guiding the integrated learning process. Simple feature fusion methods such as concatenation or weighted summation of outputs from two dimensions may introduce redundant information. However, these methods lack the ability to abstract high-level features and potentially suffer from the curse of dimensionality. To address these issues, we propose the MHMIFF. Specifically, we briefly describe the basic components of the attention mechanism and provide the detailed design of MHMIFF.

The attention mechanism allows the model to dynamically focus on important parts of the input sequence during processing. Given that $Q, K, V \in \mathbb{R}^{B\times T\times F}$ represent the query, key, and value, respectively, the attention score is computed as follows:

$$\tau_i = \frac{\sum_{j=1}^{T} \exp(Q_i^T K_j)}{\exp(Q_i^T Q_i)} \tag{14}$$

where Q_i and K_j represent the ith query sequence and jth key, respectively, B is the batch size, τ_i is the attention score for the ith query, and the weighted output is computed by applying the attention scores to the ith values V_i:

$$Atten_{out} = \sum_{j=1}^{T} \tau_i V_i \tag{15}$$

Based on the design above, the self-attention mechanism satisfies that Q, K, and V are the same. The multi-head self-attention mechanism uses multiple attention heads to perform independent attention calculations for different queries, keys, and values, as follows:

$$M_i^q = QW_i^q \qquad M_i^k = KW_i^k \qquad M_i^v = VW_i^v \quad (1 \le i \le h) \tag{16}$$

where W_i^q, W_i^k, and W_i^v represent learnable linear transformation matrices for the query, key, and value, respectively. h is the number of attention heads and M_i^q, M_i^k, and $M_i^v \in \mathbb{R}^{B\times T\times F//2h}$ are the temporary outputs of the ith attention head. Subsequently, the attention of each head is computed in parallel.

$$Emb_i = Softmax\left(\frac{M_i^q \left(M_i^k\right)^T}{\sqrt{d_k}}\right) M_i^v \tag{17}$$

where d_k is the scaling factor that prevents excessively large dot products, and $Emb_i \in \mathbb{R}^{B\times T\times F//2h}$ is the embedding representation output of the ith attention head. Finally, the outputs from all the attention heads are concatenated and passed through a linear transformation as follows:

$$Emb = f_{concat}\left(Emb_1, \ldots, Emb_h\right) W_O \tag{18}$$

where f_{concat} represents the concatenation function, W_O is the weight matrix for the final linear transformation of the output, and $Emb \in \mathbb{R}^{B\times T\times F//2}$ is the concatenated output of the multiple attention heads.

In MFFTD, mutual information is defined as the degree of information sharing between spatial features and temporal features. The amount of information shared, $I(A; B)$, can be described as follows.

$$I(A; B) = H_e(A) + H_e(B) - H_e(A, B) \tag{19}$$

where $H_e(A)$ and $H_e(B)$ represent the entropies of the spatial and temporal features, respectively, that is, their independent information content; and $H_e(A, B)$ represents the joint entropy, indicating the amount of shared information between the two. In the multi-head self-attention mechanism discussed above, the goal of mutual information is to capture the correlation between spatial and temporal features.

Building on the above discussion, we perform mutual information encoding of the spatial and temporal features to capture the bidirectional cross-domain dependencies between them. Specifically, the spatial and temporal features are denoted by In_{spa} and In_{tem}. We constructed two (Q, K, V) tuples, namely $(Q_{spa}, K_{tem}, V_{tem})$ and $(Q_{tem}, K_{spa}, V_{spa})$, to achieve mutual information encoding. The design of $(Q_{spa}, K_{tem}, V_{tem})$ is motivated by the idea that in some tasks, temporal features have a more significant impact on spatial features. In other words, time information is the more "attention-worthy" part, whereas $(Q_{tem}, K_{spa}, V_{spa})$ reflects the opposite.

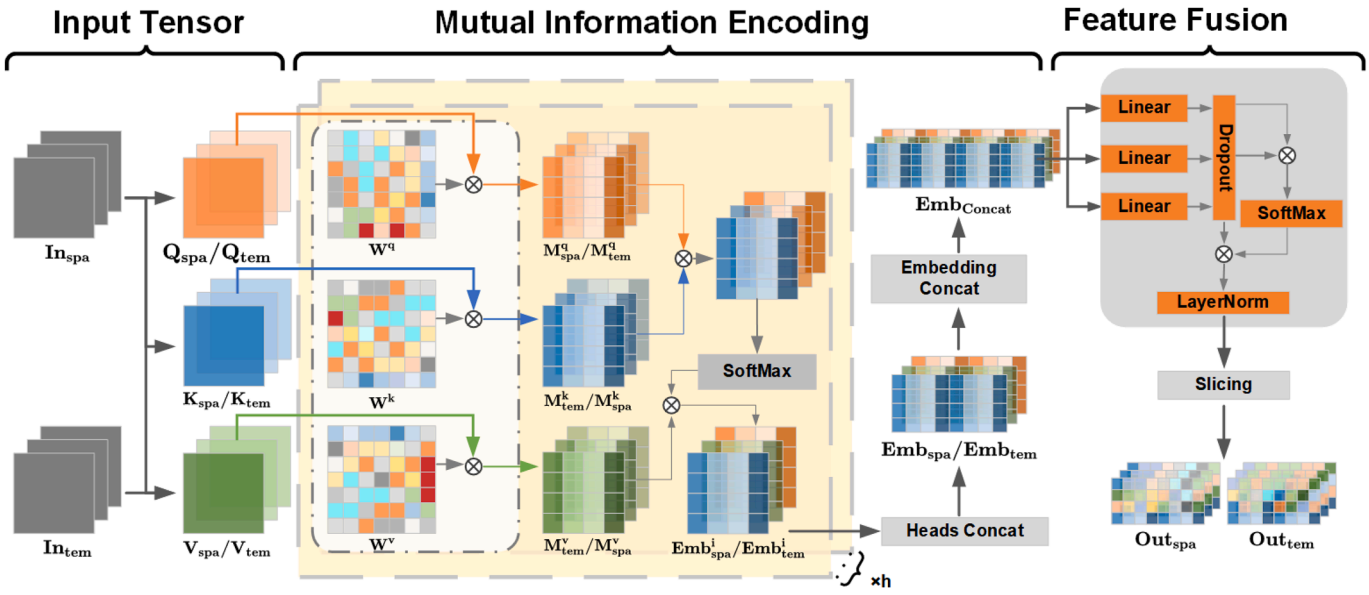


Fig. 3. Structure diagram of MHMIFF. W_i^q, W_i^k, and W_i^v represent the linear transformation weight matrices for the query, key, and value of the ith attention head, respectively. M_i^q, M_i^k, and M_i^v are the corresponding intermediate results. h denotes the number of attention heads, and Emb_i is the information encoding of the ith head. Starting from the input tensors In_{spa} and In_{tem}, the two query tuples constructed by MHMIFF are encoded via the multi-head self-attention mechanism and further utilized in the feature fusion process. Finally, the segmented output features Out_{spa} and Out_{tem} are fed into the next MFFTD layer for high-level feature extraction.

The encoded outputs are then concatenated to perform feature fusion. Similarly, a linear layer was used to enhance the generalization ability of the features, and layer normalization was applied to reduce the dependency on the batch size during training, allowing the model to focus more on expressing important features during the fusion process. Finally, the fused features Out_{spa} and Out_{tem} are used as inputs for the spatial and temporal dimensions in the next layer, respectively. Based on the above discussion, the structure of the MHMIFF is shown in Fig. 3.

In the MFFTD architecture, MHMIFF serves as the core module for deep spatiotemporal feature fusion, addressing key challenges such as redundancy and dimensional explosion. Unlike simple concatenation-based fusion methods, MHMIFF leverages a mutual information encoding mechanism to construct bidirectional cross-domain dependencies between spatial and temporal features. By quantifying shared information using Eq. (19), the model dynamically attends to informative regions while filtering out irrelevant content, effectively reducing redundancy at the source. Furthermore, the multi-head self-attention mechanism decomposes feature processing into multiple subspaces, where each head independently computes attention over the transformed features M_i^q, M_i^k, and M_i^v, enabling the parallel abstraction of high-level representations while maintaining a controlled output dimensionality. Additional refinement through linear transformation and layer normalization ensures the efficient integration of features. These mechanisms enhance the model's ability to learn complex spatiotemporal patterns, thereby improving both diagnostic accuracy and generalizability.

3.6. Loss function

In fault diagnosis models, the loss function serves as the core component linking model predictions to ground-truth labels; a rational design can enhance the model's capability to characterize complex fault patterns [42]. The following will elaborate in detail on the design logic, components of the loss function in MFFTD, and its impact on the optimization process.

In fault classification tasks, cross-entropy loss (CE) is a commonly employed metric for measuring the discrepancy between the model's predicted probability distribution and the ground-truth label distribution. Its core lies in penalizing incorrect predictions, thereby guiding the model to learn more accurate classification boundaries.

Given that the predicted output of a model for a sample (without Softmax activation) is $\hat{Y} \in \mathbb{R}^K$ (where K is the number of fault categories) and the ground-truth label is $Y_{\text{gt}} \in \{0, 1, \ldots, K-1\}$, the calculation process of cross-entropy loss is as follows:

First, the predicted output is converted into a probability distribution via the Softmax function:

$$P(k|\hat{Y}) = \frac{\exp(\hat{Y}_k)}{\sum_{i=0}^{K-1} \exp(\hat{Y}_i)} \tag{20}$$

On this basis, the cross-entropy loss is defined as:

$$\text{CE}(\hat{Y}, Y_{\text{gt}}) = -\log\left(P(Y_{\text{gt}}|\hat{Y})\right) = -\hat{Y}_{Y_{\text{gt}}} + \log\left(\sum_{i=0}^{K-1} \exp(\hat{Y}_i)\right) \tag{21}$$

In the equation, $-\hat{Y}_{Y_{\text{gt}}}$ penalizes cases where the predicted value for the true category is too low, while $\log\left(\sum_{i=0}^{K-1} \exp(\hat{Y}_i)\right)$ constrains the predicted values of all categories from increasing without bound, ultimately achieving effective penalization of incorrect predictions.

MFFTD employs a "spatial-temporal" dual-branch architecture. To achieve the collaborative optimization of spatial and temporal features, its loss function is the sum of the cross-entropy losses from the spatial branch and the temporal branch, formulated as follows:

$$\mathcal{L} = \text{CE}(Y_{\text{spat}}, Y_{\text{gt}}) + \text{CE}(Y_{\text{temp}}, Y_{\text{gt}}) \tag{22}$$

Herein, $Y_{\text{spat}} \in \mathbb{R}^K$ and $Y_{\text{temp}} \in \mathbb{R}^K$ denote the predicted outputs of the spatial branch and temporal branch, respectively, and Y_{gt} is the ground-truth label, all of which participate in the loss calculation.

Complete fault characterization requires the synergy of spatial and temporal features. Under small samples, single-dimensional features are insufficient in information, necessitating balanced utilization of both feature types to avoid information loss. In the design of the loss function, experiments demonstrate that equal weighting outperforms dynamic weighting in terms of stability, thus the equal-weight scheme is adopted.

This loss function achieves synergistic optimization of dual features through CE combination, ensuring consistent gradient directions for the two branches. It jointly drives model adjustment, avoids gradient conflicts, and not only guarantees the discriminative ability of each branch but also facilitates fusion, thereby supporting classification performance.

3.7. Time-complexity analysis

In this section, we analyze the time complexity of MFFTD based on the data flow illustrated in Algorithm 1. Given an input multivariate time-series tensor $X \in \mathbb{R}^{B\times T\times F}$, where B is the batch size, T is the time sequence length, and F is the number of input features, we break down the time complexity of each major module of MFFTD as follows:

The model initialization and fault label assignment steps are independent of the input size and are thus considered constant in time complexity: $\mathcal{O}(1)$. The time-windowing operation `sampleProduction` constructs historical sequences from raw data, resulting in complexity linear in the time dimension and window length: $\mathcal{O}(T \cdot F \cdot n_{\text{step}})$.

Each MultiScaleConv block processes input features using multiple 1D convolution kernels of different sizes. For input $X \in \mathbb{R}^{B\times F\times T}$ and kernel size k, each convolution operation has a time complexity of: $\mathcal{O}(B \cdot F \cdot T \cdot k)$. Assuming K total kernel sizes per layer, the total complexity per convolution block becomes: $\mathcal{O}(B \cdot F \cdot T \cdot \sum_{i=1}^{K} k_i)$.

Each LSTM layer processes sequences of shape (B, T, F) with hidden size H. The time complexity per LSTM layer is approximately: $\mathcal{O}(B \cdot T \cdot H^2)$. Since three LSTM layers are applied sequentially, the total complexity for the temporal module is: $\mathcal{O}(3 \cdot B \cdot T \cdot H^2)$.

MHMIFF modules perform cross-modal attention between convolutional and LSTM features. Each MHMIFF contains two `MultiHeadAttention` layers. Given input sequence length T, feature dimension F, and h heads, the attention complexity per head is: $\mathcal{O}(T^2 \cdot F/h)$. Thus, the total complexity per MHMIFF module becomes: $\mathcal{O}(2 \cdot B \cdot T^2 \cdot F)$. This is the dominant term in MFFTD due to the quadratic dependence on T, the sequence length. Three MHMIFF modules contribute: $\mathcal{O}(B \cdot T^2 \cdot F)$.

A learnable adjacency matrix $A \in \mathbb{R}^{T\times T}$ is applied multiple times to propagate correlations across variables. Matrix multiplication has a time complexity of: $\mathcal{O}(B \cdot T^2 \cdot F)$. However, due to the fixed size T, this term is typically small relative to attention and convolution operations.

The final fully connected layers map feature vectors of size D to output logits. Given layer width F, the complexity is: $\mathcal{O}(B \cdot F^2)$. Since this module is applied twice (once for spatial, once for temporal), the combined cost is: $\mathcal{O}(2 \cdot B \cdot F^2)$.

Summing the dominant components, the overall time complexity per iteration is approximated as: $\mathcal{O}(B \cdot T^2 \cdot D + B \cdot T \cdot H^2 + B \cdot F \cdot T \cdot \sum_{i=1}^{K} k_i + B \cdot T \cdot F^2)$. Here, the quadratic term $\mathcal{O}(B \cdot T^2 \cdot F)$ from the MHMIFF module dominates when T is large, due to the self-attention mechanism over time sequences. Hence, MFFTD scales efficiently with moderate input lengths but can incur higher cost with very long sequences.

4. Experiments

In this section, we conduct extensive experiments on real datasets to validate the state-of-the-art MFFTD by comparing it with classical and current mainstream models. First, we briefly introduce the dataset and dataset processing, then introduce the comparison method, and finally, we conduct extensive experiments and analysis. All experiments were conducted on a Linux server with a Python environment 3.8.18, PyTorch 2.1.1, cuda 11.8, equipped with four RTX A40 with a 46 GB GRAM graphics card. The network parameters were optimized using the Adam algorithm with a learning rate of 0.00015, epoch of 150 and a batch size of four.

Algorithm 1 The algorithmic framework of MFFTD.

Input: Time-series data $X \in \mathbb{R}^{B\times T\times F}$, labels $Y \in \mathbb{Z}^{B}$
Output: `Accuracy`, `F1`, `AUC`

$X_{\text{train}}, X_{\text{test}}, Y_{\text{train}}, Y_{\text{test}} \leftarrow$ `train_test_split`$(X, Y) \leftarrow$ `sampleProduction`(flight dataset)
Initialize model `MFFTD`
Define loss function $\mathcal{L} \leftarrow$ `CrossEntropyLoss`
Initialize optimizer *opt* $\leftarrow$ `Adam`()
for *epoch* = 1 to `Epoch` **do**
  for all batch (x, y) in training data **do**
    $F_{c1'} \leftarrow \texttt{Adjacencymatrix}_1(x)$ // Spatial Topology-dependent Capture
    $F_{c1} \leftarrow \texttt{MultiScaleConv}_1(F_{c1'})$ // Spatio Convolution Encoding
    $H_{r1} \leftarrow \texttt{LSTM}_1(x)$ // Temporal LSTM Encoding
    $F_{c1}, H_{r1} \leftarrow \texttt{MHMIFF}_1(F_{c1}, H_{r1})$ // Spatio-temporal Feature Fusion
    $F_{c2'} \leftarrow \texttt{Adjacencymatrix}_1(F_{c1})$
    $F_{c2} \leftarrow \texttt{MultiScaleConv}_2(F_{c2'})$
    $H_{r2} \leftarrow \texttt{LSTM}_2(H_{r1})$
    $F_{c2}, H_{r2} \leftarrow \texttt{MHMIFF}_2(F_{c2}, H_{r2})$
    $F_{c3'} \leftarrow \texttt{Adjacencymatrix}_1(F_{c2})$
    $F_{c3} \leftarrow \texttt{MultiScaleConv}_3(F_{c3'})$
    $H_{r3} \leftarrow \texttt{LSTM}_3(H_{r2})$
    $F_{c3}, H_{r3} \leftarrow \texttt{MHMIFF}_3(F_{c3}, H_{r3})$
    // Classification Head
    $\hat{Y}_{\text{spat}} \leftarrow \texttt{FC}_1(F_{c3})$
    $\hat{Y}_{\text{temp}} \leftarrow \texttt{FC}_2(H_{r3})$
    // Loss and Optimization
    $\mathcal{L} \leftarrow \texttt{CE}(\hat{Y}_{\text{spat}}, Y_{\text{gt}}) + \texttt{CE}(\hat{Y}_{\text{temp}}, Y_{\text{gt}})$
    `Backpropagate and update parameters`
  end for
end for
// Evaluation
$\hat{Y}_{\text{spat}}, \hat{Y}_{\text{temp}} \leftarrow \texttt{model}(X_{\text{test}})$
Compute `Accuracy`, `F1`, `AUC`
Save best-performing model based on validation accuracy
return `Accuracy`, `F1`, `AUC`

4.1. Datasets and data-processing processes

4.1.1. Datasets

The four real-world public datasets analyzed in this study, publiced by Bronz [12], were collected on various days in July 2020. Each dataset was generated from independent flight sessions, with different injected faults and variations in sensor readings and flight conditions. The public datasets was collected by Bronz [12] based on the open-source autopilot system Paparazzi, as shown in Fig. 4, which comprises three main parts: the aircraft, ground control station (GCS), and RC-transmitter. Faults are injected through the GCS while the RC-Transmitter controls the flight of the fixed-wing UAV, and the flight data generated by the UAV are transmitted back to the GCS through the X-Bee. In this flight experimental system, the actuator faults can be mathematically modeled using the following equation [43]:

$$u_{\text{app}} = d u_{\text{com}} + e \tag{23}$$

where u_{com} is the commanded control deflection from the autopilot to the actuators, u_{app} is the actual deflection performed by the vehicle, d is the efficiency of the execution of the deflection instruction, represents the fault type, and e is the error. The fault model of the two aircraft aerodynamic control surfaces can be expressed in terms of vectors as follows:

$$\begin{bmatrix} u_{app1} \\ u_{app2} \end{bmatrix} = \begin{bmatrix} d_1 & 0 \\ 0 & d_2 \end{bmatrix} \begin{bmatrix} u_{com1} \\ u_{com2} \end{bmatrix} + \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \tag{24}$$

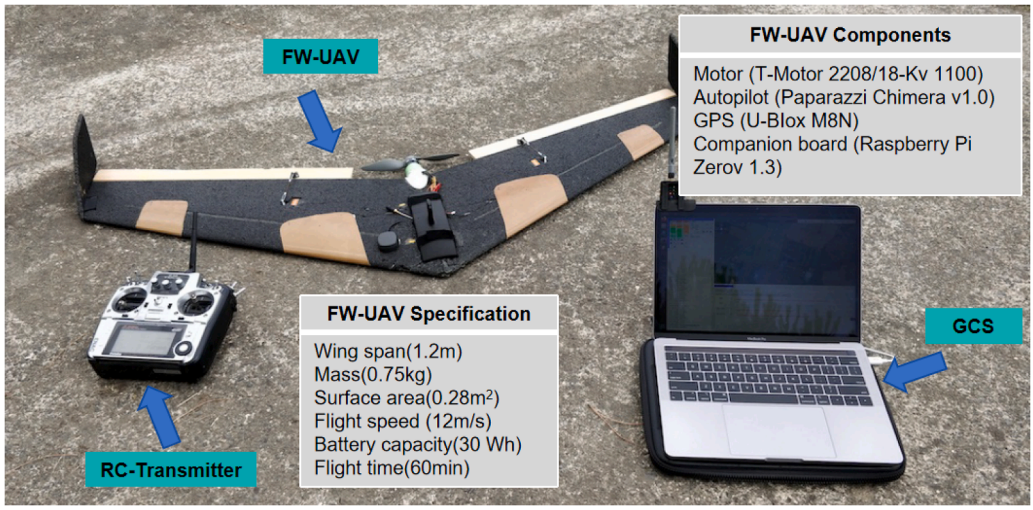


Fig. 4. Data collection system.

Table 3
Description of flight dataset.

Flight date	Wind speed	Classes	Sample sizes	Fault type → category	Categorys numbers
July 12,	<2.0 m/s	2	25900	Normal → 0, $d_1 = 0.3 \rightarrow 1$	0: 18216 1:7684
July 13,	8.0 m/s	2	29430	Normal → 0, $d_1 = 0.3 \rightarrow 1$	0: 21035 1:8395
July 21,	2.5 m/s	9	29790	Normal → 0, $d_1 = 0.3 \rightarrow 1$, $d_2 = 0.9 \sim 0.3 \rightarrow 2 \sim 8$	0:14712 1:2417 2:1473 3:1491 4:1489 5:1556 6:1531 7:1612 8:3509
July 23,	5.0 m/s	9	32240	Normal → 0, $d_1 = 0.3 \rightarrow 1$, $d_2 = 0.9 \sim 0.3 \rightarrow 2 \sim 8$	0:16929 1:2523 2:1426 3:1458 4:1479 5:1503 6:1547 7:1615 8:3760

where the subscripted 1 parameter represents the right wing of the vehicle, and the subscripted 2 parameter is for the left wing of the vehicle.

We selected July 12, 13, 21, and 23 flight data, as shown in Table 3 for our experiments, where the flight data for the 12th and 13th contain normal data and one type of malfunction data for the right wing d1 = 0.3. and the data for the 21st and 23rd contain normal data, eight types of malfunctions for the right wing d1 = 0.3 and left wing d2 = 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3. We also provided the sample size for each category, which clearly indicated the presence of a class imbalance problem in the dataset.

4.1.2. Data-processing processes

Raw flight data were recorded at a sampling frequency of 50 Hz for 50 variables. Because of the inconsistency in the sampling rates among different variables, a direct application for fault diagnosis is not feasible. We employed linear interpolation to standardize the sampling rate of all variables to 10 Hz, aligning the data while reducing both the data volume and computational burden. Typically, decreasing the number of irrelevant features can lower the computational costs and enhance the model performance. As shown in Fig. 5, we select 12 representative variables : airspeed (flight speed), phi (roll angle), psi (yaw angle), theta (pitch angle), Ax, Ay, Az (linear accelerations along the x, y, and z axes), Gx, Gy, Gz (angular velocities along the x, y, and z axes), and C1, C2 (control commands for the right and left wings). Next, to reduce the bias towards specific features and enhance model performance, we applied Z-score normalization Eq. (25) to the dataset, as illustrated in Fig. 6. The areas marked in red in Fig. 6 is the fault data.

$$X_{scale} = \frac{X - \mu}{\sigma} \tag{25}$$

where μ is the mean, and σ is the standard deviation.

After processing the dataset, samples were prepared starting from 20 time steps, as shown in Fig. 7. The features at time t are

$$\begin{aligned} x(t) &= [x_t^1, x_t^2, \ldots, x_t^{12}] \\ x(t-1) &= [x_{t-1}^1, x_{t-1}^2, \ldots, x_{t-1}^{12}] \\ &\vdots \\ x(t-19) &= [x_{t-19}^1, x_{t-19}^2, \ldots, x_{t-19}^{12}] \end{aligned} \tag{26}$$

The features of the sample at time t are as follows:

$$X(t) = [x(t), x(t-1), \ldots, x(t-10), \ldots, x(t-19)]$$

$$X(t) = \begin{bmatrix} x_t^1 & \cdots & x_t^7 & x_t^8 & \cdots & x_t^{12} \\ x_{t-1}^1 & \cdots & x_{t-1}^7 & x_{t-1}^8 & \cdots & x_{t-1}^{12} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \\ x_{t-19}^1 & \cdots & x_{t-19}^7 & x_{t-19}^8 & \cdots & x_{t-19}^{12} \end{bmatrix} \tag{27}$$

4.2. Methods for comparison

In this section, we review the relevant literature and select state-of-the-art and classical methods based on traditional machine learning or neural networks to comprehensively evaluate the performance of MFFTD. The benchmarks used are summarized below.

1. **SVM** [12]: SVM is a classical machine learning algorithm where the goal is to find a hyperplane that completely separates the different classes of data points and maximizes the distance of the two classes of data points to this hyperplane.
2. **CANN** [44]: CANN is an attention-based convolutional neural network model with multiple spatial and channel attention modules.
3. **SHNN** [16]: SHNN is a few-shot learning approach to address FW-UAV fault diagnosis by employing multiple series-connected CNN and LSTM networks to capture the spatio-temporal features of flight data.

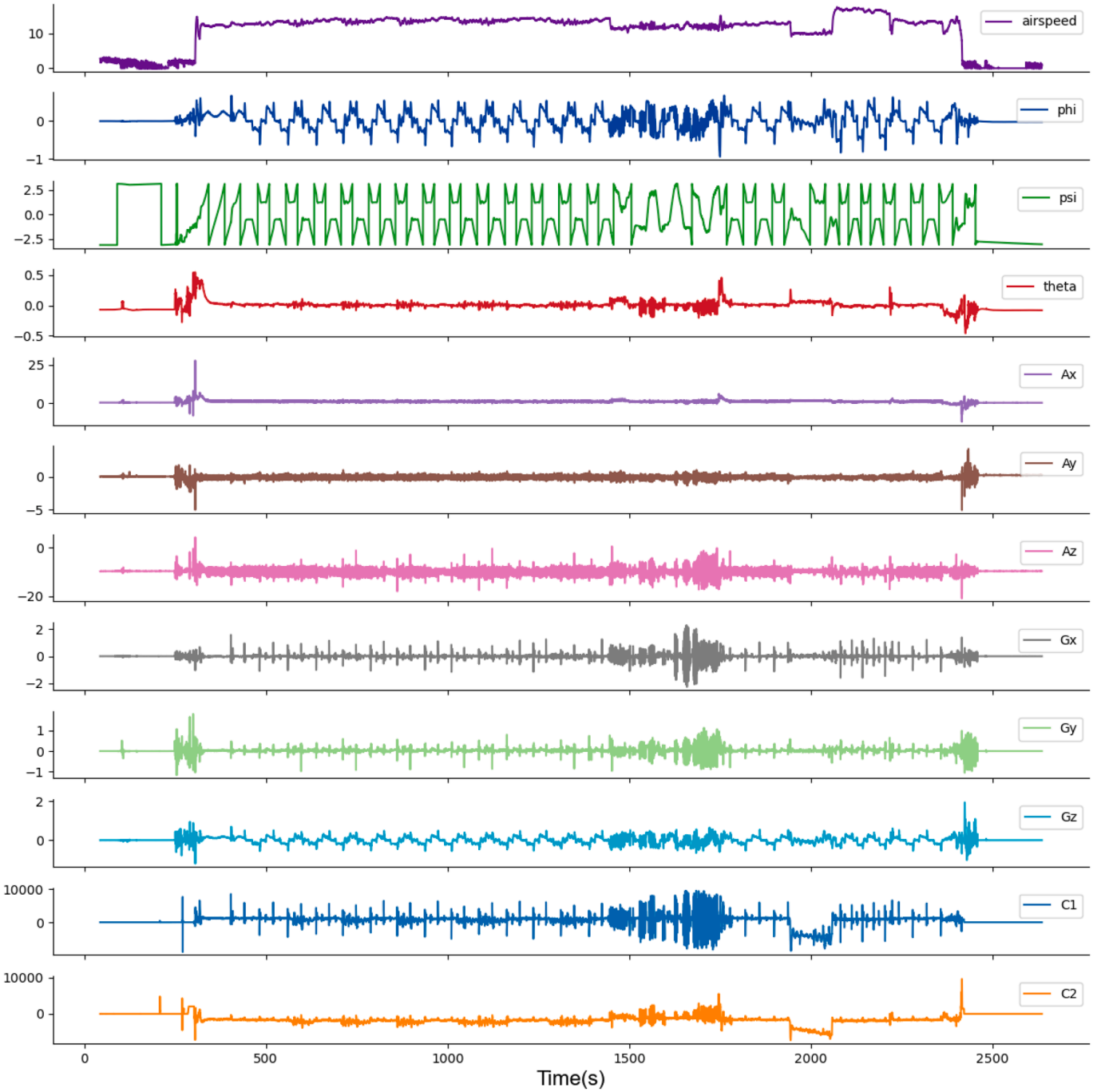


Fig. 5. Visualization of the 12 selected features in the dataset of July 21st.

4. **ASN** [36]: ASN is a few-shot learning neural network that utilizes two parallel feature extractors with shared weights and a channel attention mechanism to adaptively trade-off features from different channels and reduce the coupling effect of multi-channel data.
5. **NDDR-CNN** [45]: NDDR-CNN is a CNN architecture for multi-task learning that effectively fuses inter-layer features through a neural discriminant dimensionality reduction module, achieving superior outcomes in multi-task learning scenarios.
6. **Cross-Stitch** [46]: Cross-Stitch is a well-known generalized multi-task learning approach. Networks with a "cross-stitch" unit can learn the optimal combination of shared and task-specific representations.
7. **HDMTL** [17]: HDMTL is the current latest research framework. It captures the spatio-temporal characteristics of flight data by using sub-networks and transfers knowledge among sub-networks with the help of attention strategies to achieve mutual complementarity.

To ensure fairness in the comparison, we used the best evaluation results and recommended hyperparameter settings from the original studies. Notably, the parameter settings of the ASN and SHNN models follow those in the original study and adopt the "N-way K-shot" setting. We use the five-shot parameters of the original paper for comparison, and the NDDR-CNN, Cross-Stitch, HDMTL and our MFFTD all had temporal and spatial channels for the experiment. As before, we selected the accuracy

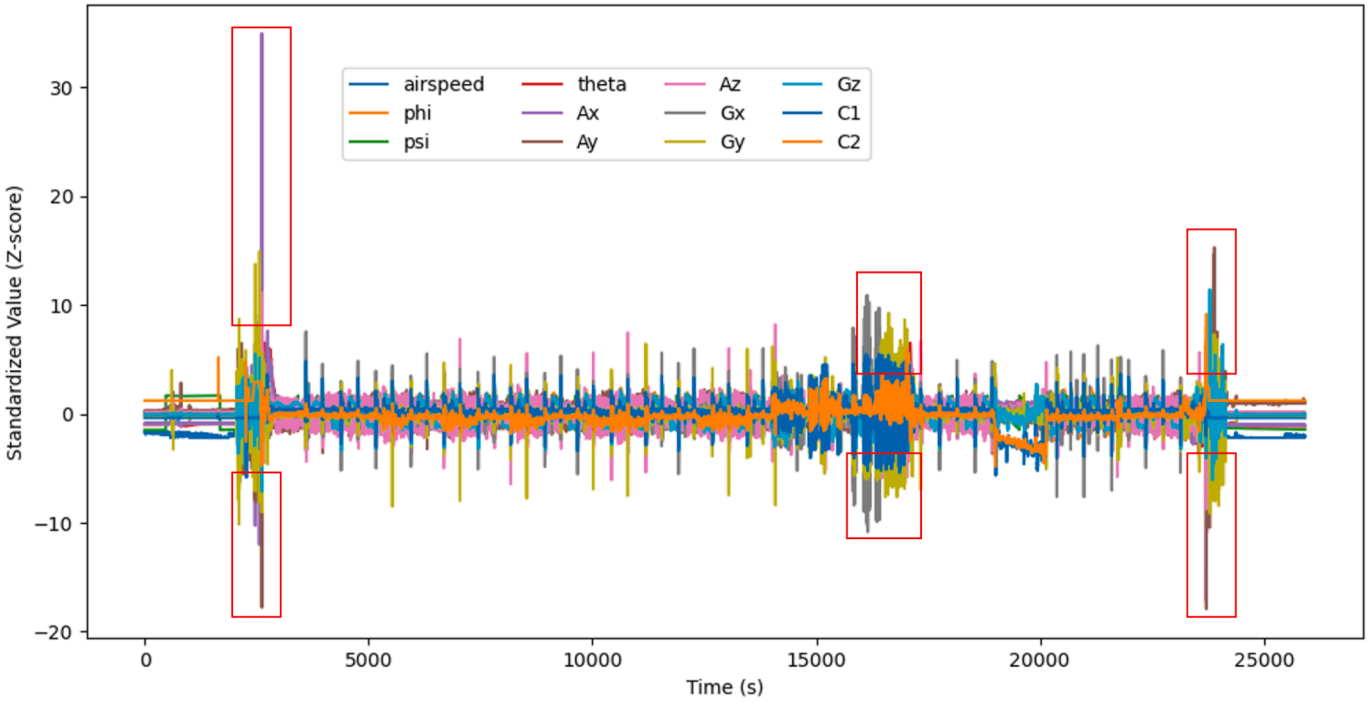


Fig. 6. Data normalization.

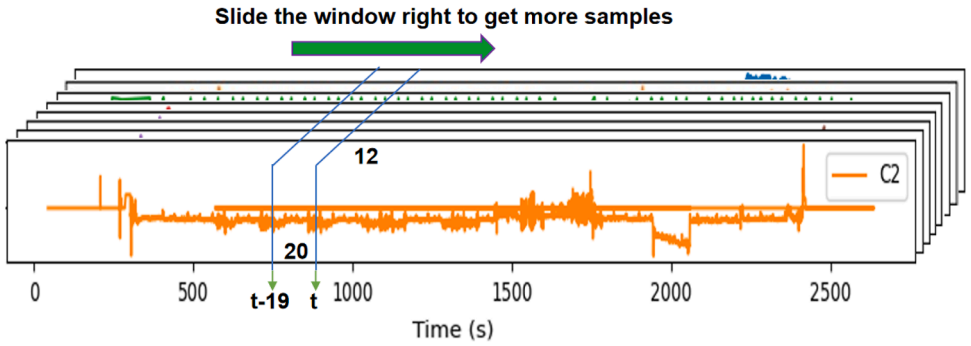


Fig. 7. Sample production.

rate as a measure of the model, which is calculated as follows:

$$Accuracy = \frac{C}{T} \times 100\,\% \tag{28}$$

Here, C is the number of correctly categorized samples, and T is the number of full samples.

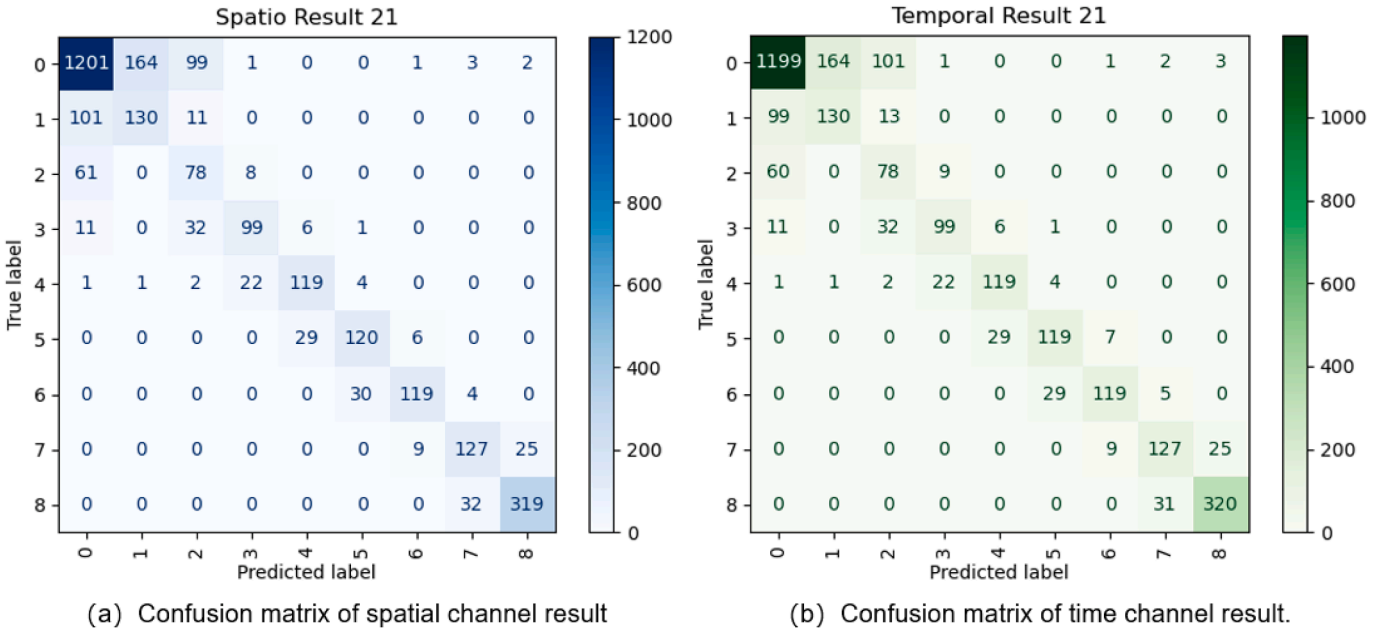
4.3. Fault diagnosis under small samples

In this section, considering that the fault data sample size of drones is extremely limited in real life, to fully demonstrate the advantage of our constructed model for fault diagnosis, we conducted a series of experiments and analyses with a small number of samples. First, owing to the imbalance in the dataset, an equal number of samples were randomly selected for each fault category from the flight data on July 12, 13, 21, and 23, yielding 90 samples for model training. Subsequently, 10 % of the overall data was selected for testing. It should be emphasized that there were no repetitions of samples in the training and test sets, and the test set represented the entire dataset. Finally, we conducted extensive experiments,with each experiment repeated ten times and the results averaged, and the relevant experimental results and model comparisons are presented in Table 4. The best results are shown in bold and the second-best results are underlined.

Table 4
Experiment results of different models with small samples.

Models	12-Jul	13-Jul	21-Jul	23-Jul	Average
SVM	88.11	84.76	36.96	36.96	61.70
CANN	85.46	84.72	31.87	28.66	57.68
SHNN(5-shot)	89.38	88.36	46.80	41.14	66.42
ASN(5-shot)	88.16	87.37	53.65	42.40	67.90
NDDR CNN-S	79.87	80.66	36.87	38.25	58.91
NDDR CNN-T	80.17	82.21	35.17	33.30	57.71
Cross Stitch-S	79.01	70.40	16.88	27.96	48.56
Cross Stitch-T	**91.09**	87.25	46.63	47.26	68.06
HDMTL-S	88.64	91.31	66.83	57.37	76.04
HDMTL-T	89.64	**91.42**	66.65	57.59	76.33
MFFTD-S	88.03±1.07	89.24±1.23	73.09±1.94	**64.35±2.70**	78.68±1.85
MFFTD-T	88.10±1.32	89.32±1.44	**73.33±2.32**	65.38±2.35	**79.03±1.92**

Based on the experimental results with small sample sizes, MFFTD outperformed the other models for the entire dataset. In particular, for the multi-classification tasks in the datasets from July 21 to 23, MFFTD showed a significant advantage, that it increased the fault classification accuracy in multi-classification tasks by 8% compared to the latest models. Owing to its unique design, MFFTD can adapt better to different data characteristics and task requirements. HDMTL also performed



(a) Confusion matrix of spatial channel result (b) Confusion matrix of time channel result.

Fig. 8. Visualization of multi-classification results (July 21st dataset): Fig. a (spatial channel), Fig. b (time channel).

well, ranking first in the binary classification tasks for the datasets from July 12 and 13, and second to MFFTD in the multi-classification tasks. Compared with MFFTD, HDMTL's feature extraction network of HDMTL relies solely on a simple convolutional network and LSTM, and its knowledge complementary module only performs simple concatenation of spatiotemporal features, which causes information redundancy and results in lower classification performance than MFFTD. Thus, there is room for improvement in the feature extraction and fusion aspects of HDMTL. Despite the temporal excellence of Cross-Stitch, its sharing mechanism inadequately coordinates spatiotemporal channel interactions, underutilizing temporal information in spatial channels and worsening feature imbalance. In contrast, MFFTD achieves comprehensive feature integration across both dimensions through its unique sharing mechanism. The dual-channel concatenation design of NDDR-CNN lacks sufficient interaction, making it difficult to capture critical spatiotemporal features. Moreover, the meta-learning approaches used in SHNN and ASN achieve better fault diagnosis results than the conventional deep learning methods, SVM and CANN, although they are still inferior to diagnosis methods with spatiotemporal feature-sharing mechanisms. The conventional deep learning methods, SVM and CANN perform the worst in fault diagnosis owing to the limited number of samples.

To demonstrate the efficacy of MFFTD in multi-classification tasks more effectively, we visualized the test results of the dataset on July 21st, as shown in Fig. 8. Specifically, Fig. 8(a) presents the results of the spatial channels, while Fig. 8(b) displays the results of the time channels. It can be seen from the confusion matrix that MFFTD exhibits excellent performance in the tests of the spatial and temporal channels. The values on the main diagonal were significantly higher than those at the other positions, indicating a relatively high classification accuracy. Moreover, the samples that were misclassified by the model were mainly concentrated among neighboring categories, and the distributions of these samples were quite close. This demonstrates that MFFTD possesses an outstanding ability to collaboratively learn spatial-temporal features, enabling it to effectively capture both static and dynamic characteristics, adapt to complex scenarios, and achieve remarkable performance in multi-classification tasks.

In addition, we visualized the training process of MFFTD on 90 samples across the four datasets, as illustrated in Fig. 9. In the figure, the solid lines represent the diagnostic process of the spatial channel and the dashed lines correspond to the diagnostic results of the temporal channel. Remarkably, we observed a high degree of accuracy overlap between the temporal and spatial channels during training. This reflects the strong coordination exhibited by the MFFTD model in spatiotemporal feature extraction and fusion. The MHMIFF effectively enables deep fusion and complementary feature learning. Such a collaborative design not only enhances the model's capability to represent complex patterns but also significantly improves the stability and generalization performance of the diagnostic system.

4.4. Fault diagnosis under different samples

When the number of training samples was 90, MFFTD achieved excellent results. To more fully evaluate the performance of the MFFTD, in this section, we randomly selected a small number of training samples of different quantities for each dataset, including 90, 120, 150, 180, 270, 360, 720, and 1440. In particular, considering the class imbalance in the dataset, we ensured that the number of samples for each category remained consistent for different training sample sizes. Subsequently, 10 % of the dataset was selected as the test set. Subsequently, we performed extensive binary and multi-class classification experiments, with each experiment repeated ten times and the results averaged, and adopted a multi-dimensional evaluation framework for the analysis of the results. In addition to the traditional accuracy metric, we included Macro-F1 and AUC-ROC as key evaluation indicators. Both Macro-F1 and AUC-ROC were calculated as the averages of the spatial and temporal channels. Tables 5 to 8 respectively record the fault diagnosis results of the different models on the July 12, 13, 21, and 23 datasets. The best results are shown in bold and the second-best results are underlined.

According to the results of the binary classification of the dataset on July 12 and 13 in Tables 5 and 6, all methods achieved remarkable results. When the number of training samples was between 90 and 180, HDMTL exhibited the best performance, followed by MFFTD. Moreover, MFFTD demonstrates a significant advantage in both AUC-ROC and Macro-F1 metrics. For instance, in the dataset from July 12, with a sample size of 180, an AUC of 95.84 and a Macro-F1 score of 91.38. These results not only validate its capability in extracting critical features from limited data but also highlight its effectiveness in distinguishing between positive and negative samples and ensuring classification fairness under class-imbalanced scenarios. The ASN and SHNN few-shot learning models closely followed. However, owing to the

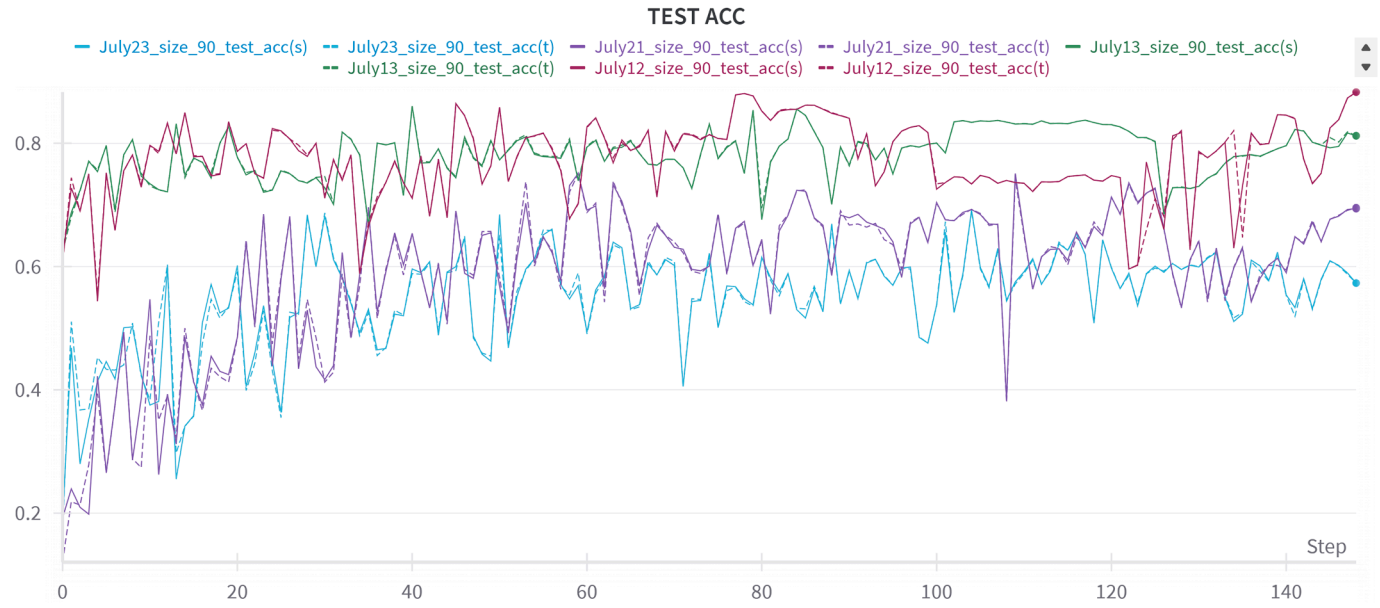


Fig. 9. Visualization of the training process of MFFTD.

Table 5
Model comparison with different small training samples on July 12 dataset.

Models	Sample sizes (n)							
	90	120	150	180	270	360	720	1440
SVM	88.11	89.00	90.00	90.44	92.33	93.56	95.00	97.22
CANN	85.46	88.23	89.65	90.83	93.19	94.52	96.64	98.53
ASN(5-shot)	88.16	91.82	92.68	92.06	95.30	94.86	89.40	92.76
SHNN(5-shot)	89.38	91.31	92.87	**93.76**	**95.82**	95.98	97.61	97.04
NDDR-CNN-S	79.87	83.78	88.58	86.87	90.67	93.52	93.84	95.32
NDDR-CNN-T	80.17	82.71	87.23	85.76	87.30	90.86	91.00	93.80
Cross Stitch-S	79.01	85.51	86.92	89.07	93.54	95.10	97.61	98.90
Cross Stitch-T	**91.09**	89.61	90.16	91.70	95.61	**96.68**	**98.00**	**99.11**
HDMTL-S	88.64	**92.03**	**92.93**	92.60	94.49	95.89	96.60	98.61
HDMTL-T	89.64	92.01	92.85	92.60	94.49	95.86	96.58	98.61
MFFTD-S	88.03±1.07	90.38±1.04	91.58±1.40	92.74±0.99	94.71±0.93	95.40±0.73	97.74±0.53	98.92±0.44
MFFTD-T	88.10±1.32	90.69±1.05	91.66±1.31	92.93±1.02	94.94±0.93	95.50±0.77	97.72±0.66	98.83±0.53
AUC	88.23±2.54	89.91±2.31	92.21±1.83	95.84±1.04	95.30±1.34	96.28±1.29	98.10±1.40	97.31±1.13
Macro-F1	84.34±1.45	87.31±1.05	88.33±2.05	91.38±1.20	93.24±1.08	94.61±0.82	96.86±1.05	96.25±0.97

extremely small number of samples, Cross Stitch has a large disparity in the effects of the time and space channels. As the number of samples continues to increase, the accuracy of all models significantly improves. Among them, the effects of ASN and SHNN were astonishing. The cross-stitch compensates for the deficiency of insufficient samples and reaches a leading level. While maintaining an advantage in accuracy, MFFTD also sustained a high performance in both AUC-ROC and Macro-F1 metrics. For example, on the July 12, dataset with a sample size of 1440, it achieved an AUC of 97.31 and a Macro-F1 score of 96.25, indicating a stable and well-rounded performance. Nevertheless, a simple binary classification task cannot evaluate the performance of the models better.

Based on the results of the multiclass classification tasks from the datasets on July 21 and 23, presented in Tables 7 and 8, it is evident that all methods exhibit a significant decrease in accuracy compared with binary classification tasks, indicating that multiclass fault diagnosis is inherently more complex and diverse. Among these, MFFTD consistently outperformed the other models for different sample sizes. Its advantage is especially evident with smaller sample sizes (90, 120, 150, and 180), demonstrating strong performance under data scarcity. As the sample size increases, models such as Cross-Stitch and HDMTL improve significantly, whereas few-shot models (ASN and SHNN) and traditional deep learning models (SVM and CANN) struggle with low accuracy and fail to adapt to the complexity of multiclass scenarios. Furthermore, MFFTD not only maintains high accuracy, but also achieves outstanding AUC-ROC scores of 99.57 and 99.28, and Macro-F1 scores of 95.32 and 91.23, July 21 and 23 datasets, respectively. These metrics underscore their robustness to decision thresholds and their excellent performance in imbalanced multiclass scenarios, particularly in recognizing minority classes.

To clearly demonstrate the fault diagnosis capability of MFFTD in multi-classification tasks, we applied t-SNE to the datasets from July 21 to 23, before and after training, with sample sizes of 90 and 1440, respectively, as shown in Fig. 10. The left column shows the initial distribution of the test sets where the categories were intermingled, indicating that the data's manifold structure was not effectively revealed before training. With a sample size of 90 (middle column), MFFTD exhibits strong classification performance, clustering the sample points and revealing the latent manifold structure through its multi-scale feature fusion mechanism. As the sample size increased to 1440 (right column), the classification accuracy further improved, with the categories completely separated into distinct clusters. This indicates that MFFTD can fully explore the complex manifold structure of the data and enhance the classification accuracy as the sample size increases.

Table 6
Models comparison with different small training samples on July 13 dataset.

Models	Sample sizes (n)							
	90	120	150	180	270	360	720	1440
SVM	84.76	88.88	90.66	**93.55**	93.88	94.22	95.33	97.00
CANN	84.72	88.15	87.98	91.69	92.74	92.49	97.10	97.62
ASN(5-shot)	87.37	90.30	90.56	91.69	93.39	**95.86**	97.48	98.63
SHNN(5-shot)	88.36	89.66	91.37	91.35	93.05	96.30	**97.80**	98.22
NDDR-CNN-S	80.66	82.50	84.75	87.96	89.21	90.53	92.45	93.09
NDDR-CNN-T	82.21	81.06	83.18	86.13	87.20	88.51	91.43	94.61
Cross Stitch-S	70.40	79.59	83.59	89.58	95.29	95.35	97.93	99.00
Cross Stitch-T	87.25	87.20	89.42	92.22	**95.63**	95.58	97.60	**99.02**
HDMTL-S	91.31	**92.21**	92.73	91.13	94.87	95.43	96.91	98.82
HDMTL-T	**91.42**	92.14	**92.81**	91.00	94.81	95.43	96.84	98.85
MFFTD-S	89.24±1.55	90.72±1.23	91.23±1.28	92.35±1.10	94.33±1.42	95.68±0.71	96.44±0.85	98.42±0.94
MFFTD-T	89.32±1.39	90.72±1.44	91.30±1.19	92.29±1.11	94.41±1.45	95.58±0.76	96.53±0.88	98.36±0.87
AUC	86.42±3.12	88.17±2.66	91.74±2.52	92.68±1.38	91.87±2.50	93.32±2.27	94.27±1.80	98.85±0.60
Macro-F1	84.02±1.20	84.78±1.62	87.11±1.26	88.56±1.21	87.99±1.66	89.95±0.97	92.10±1.14	95.73±1.17

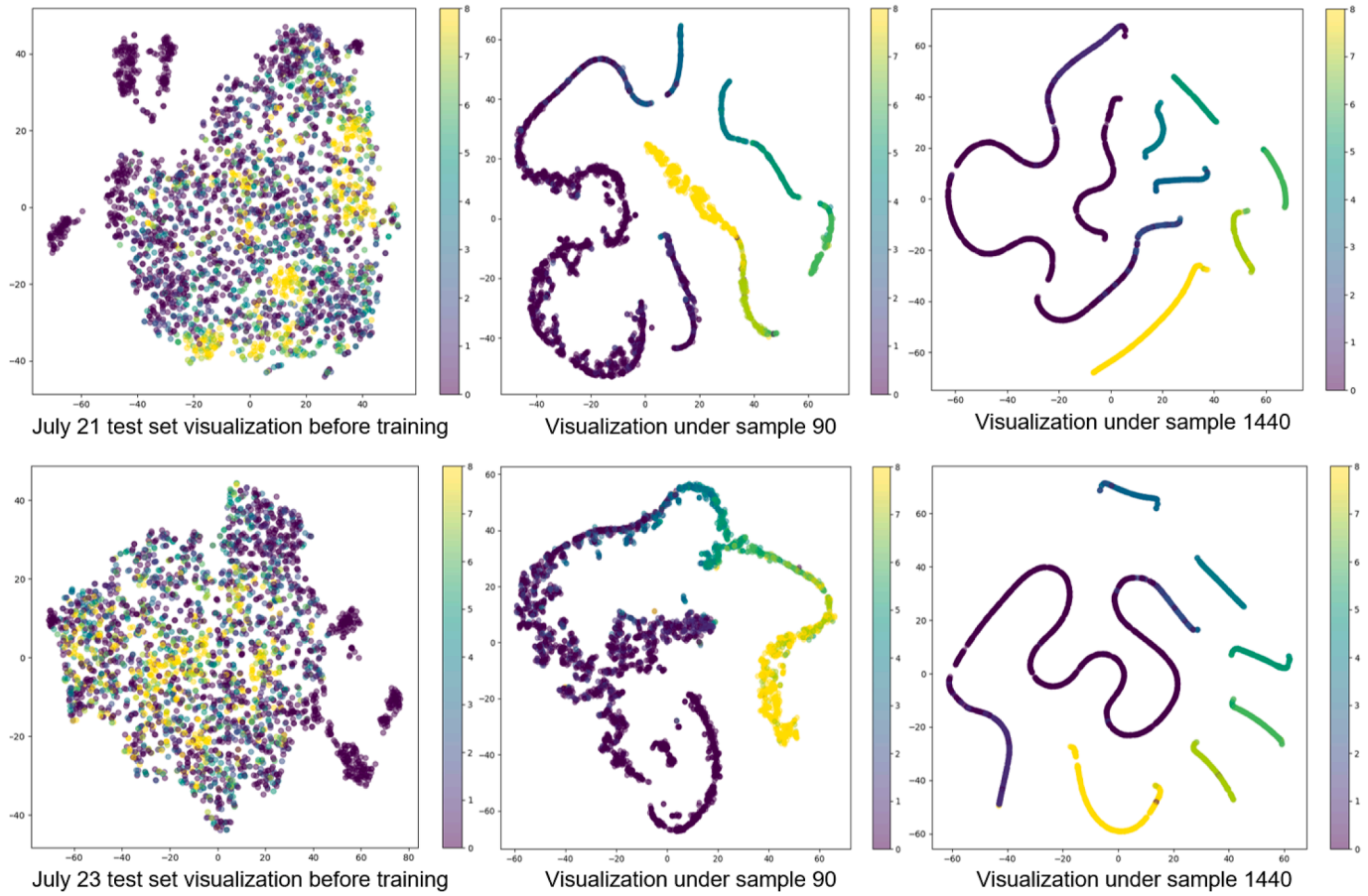


Fig. 10. The t-SNE visualization of the features in multi-classification tasks.

This chapter demonstrates the MFFTD's classification performance via multi-dimensional comparative experiments (including varying sample sizes, multi-source datasets, and diverse tasks) and visual analytics. Compared with existing models, MFFTD shows superior generalization under complex conditions, specifically:

1. Cross-time-series: Using four real flight trajectory sensor data shown in Table 3, MFFTD performs stably across diverse time-window sequences.
2. Cross-scenario: Across four real datasets spanning wind conditions and flight trajectories, MFFTD maintains high fault recognition accuracy despite varying wind and sensor noise.
3. Cross-distribution feature: MFFTD retains strong performance amid fault probability distribution discrepancies across dates and diverse classification tasks.

4.5. Hyperparameter optimization

We conducted a sensitivity analysis on the main hyperparameters of MFFTD, including the learning rate (lr) and batch_size. These hyperparameters are critical factors that influence the evaluation performance of MFFTD. The convolutional output channels and dropout rates are determined based on an empirical design. Specifically, the convolutional

Table 7
Model comparison with different small training samples on July 21 dataset.

Models	Sample sizes (n)							
	90	120	150	180	270	360	720	1440
SVM	36.96	41.58	47.18	50.64	58.76	63.12	70.48	78.79
CANN	31.87	35.59	37.10	39.38	49.46	54.24	70.13	81.83
ASN(5-shot)	53.65	59.18	62.22	58.49	59.35	60.35	63.22	66.67
SHNN(5-shot)	46.80	55.91	51.77	56.88	58.15	57.96	60.69	60.35
NDDR-CNN-S	36.87	39.53	43.44	46.99	49.60	56.02	63.98	72.61
NDDR-CNN-T	35.17	36.79	40.17	44.12	46.06	52.73	62.27	72.19
Cross Stitch-S	16.88	19.98	29.36	38.74	63.48	75.34	89.98	93.70
Cross Stitch-T	46.63	51.45	55.52	61.13	82.83	84.19	91.90	94.23
HDMTL-S	66.83	70.50	73.53	77.76	82.94	84.06	89.97	92.67
HDMTL-T	66.65	70.02	73.69	77.63	83.01	83.91	90.04	92.77
MFFTD-S	73.09±1.94	**77.52±1.63**	**80.26±1.55**	**82.65±1.05**	**84.66±0.90**	86.73±0.97	92.31±0.56	**95.02±0.66**
MFFTD-T	**73.33±2.34**	77.46±1.80	80.17±1.55	82.62±0.97	84.63±0.70	**86.81±0.88**	**92.36±0.58**	94.72±0.62
AUC	94.29±0.90	95.64±0.81	97.20±0.97	96.86±0.85	98.16±0.26	98.09±0.62	99.21±0.17	99.57±0.14
Macro-F1	68.80±2.32	75.23±2.37	80.06±1.57	79.61±1.24	85.37±1.32	86.70±1.51	93.24±1.35	95.32±0.64

Table 8
Model comparison with different small training samples on July 23 dataset.

Models	Sample sizes (n)							
	90	120	150	180	270	360	720	1440
SVM	36.96	41.58	40.92	46.96	54.38	55.18	63.48	74.01
CANN	28.66	31.64	34.29	40.32	44.25	46.44	60.99	74.27
ASN(5-shot)	42.40	47.36	49.76	49.78	51.48	52.06	54.65	55.96
SHNN(5-shot)	41.14	47.50	45.99	46.26	46.60	47.32	48.28	50.14
NDDR-CNN-S	38.25	41.77	42.24	47.64	49.44	52.74	60.87	69.98
NDDR-CNN-T	33.30	35.93	39.35	42.84	44.69	48.69	60.80	68.98
Cross Stitch-S	27.96	33.39	33.85	44.68	51.25	64.27	84.94	92.47
Cross Stitch-T	47.26	50.25	52.49	61.76	69.39	74.89	87.30	**93.44**
HDMTL-S	57.37	62.71	67.71	70.74	73.56	71.51	83.07	90.20
HDMTL-T	57.59	62.54	67.61	70.61	73.54	71.64	82.99	90.00
MFFTD-S	64.35±2.78	**69.63±2.46**	71.32±2.21	72.53±1.77	78.76±1.94	**80.41±1.28**	86.12±1.16	92.78±1.08
MFFTD-T	**65.38±2.89**	69.52±2.44	**71.41±2.13**	**72.57±1.71**	**78.81±1.97**	80.36±1,35	**86.14**±1.07	92.66±0.96
AUC	91.40±1.78	93.36±1.03	93.11±1.37	94.47±0.92	94.54±1.30	96.47±1.22	98.67±1.66	99.28±1.43
Macro-F1	58.48±3.14	61.10±3.26	62.05±2.86	65.73±2.47	65.80±1.84	73.76±1.73	85.89±1.63	91.23±1.26

output channels (in_dim, hidden_dim, out_dim) were set to 128, 256, and 512, respectively, while the dropout rate was consistently maintained at 0.2. To investigate the internal factors influencing MFFTD under small-sample conditions, we conducted hyperparameter experiments using datasets from July 21 and 23, each comprising 90 samples. Each experiment was repeated ten times, and the average result was used for the analysis. The experimental results are presented in Table 9.

Analysis based on hyperparameter experimentation indicated that the learning rate and batch size significantly influenced the performance of the MFFTD model under few-shot conditions. Regarding the learning rate, setting $lr = 0.00015$ yielded an overall superior performance. This configuration aligned well with the internal parameter update mechanism of the model and the feature extraction demands of convolutional channels (ranging from 128 to 512 dimensions), ensuring effective spatiotemporal feature extraction while maintaining optimization stability. This balance contributed to both efficient convergence and stable feature learning. In terms of the batch size, $b = 4$ consistently outperformed the other settings across multiple experiments. Under few-shot training scenarios, this batch size effectively balanced the gradient updates and sample utilization. This mitigates the gradient noise typically observed in smaller batches (e.g., $b = 2$) and avoids the inefficiency of larger batches (e.g., $b = 6$) by utilizing limited samples, thereby optimizing the training mechanism of the model.

4.6. Ablation experiments

To verify the effectiveness of the key module of MFFTD, ablation experiments were conducted by selecting 90 and 120 samples from the four datasets. The experimental results are presented in Table 10. This experiment was conducted ten times and the average value was recorded. The specific settings of the ablation experiment were as follows.

1. We removed the multi-head mutual information self-attentive feature fusion module to make the time channel and the spatial channel independent of each other for fault diagnosis respectively.
2. We removed the temporal dynamic graphs, and only the multi-scale residual convolutional network was left for the learning of the spatial channel.
3. We replaced the multi-scale residual convolutional network (with convolution kernels of 3, 5, 7, and 9) with ordinary one-dimensional convolution (with a convolution kernel of 3).
4. Completed MFFTD.

From the results of the ablation experiment, the independent detection of the spatial and temporal channels with no feature fusion had the lowest accuracy rate, and the gap between the spatial and temporal channels was considerably large. Compared with the complete MFFTD, the multi-head mutual information self-attentive feature fusion module

Table 9
Hyperparameter experiment results.

Models	$lr = 0.00001$			$lr = 0.00005$			$lr = 0.0001$			$lr = 0.00015$			$lr = 0.0002$		
	$b = 2$	$b = 4$	$b = 6$	$b = 2$	$b = 4$	$b = 6$	$b = 2$	$b = 4$	$b = 6$	$b = 2$	$b = 4$	$b = 6$	$b = 2$	$b = 4$	$b = 6$
MFFTD(21S)	65.77±2.89	63.91±2.78	61.34±3.22	68.43±3.45	66.44±3.18	67.64±2.55	72.89±3.33	71.11±2.05	68.27±0.97	**73.47±1.80**	73.26±2.20	68.67±2.46	71.30±3.44	72.64±1.90	72.80±2.88
MFFTD(21T)	66.04±2.51	63.77±2.39	61.12±2.97	68.80±3.16	66.56±3.13	67.66±2.56	72.32±2.72	70.91±2.20	68.18±0.83	**73.68±1.79**	73.29±2.44	69.04±2.31	71.03±3.72	72.87±2.23	71.32±2.93
MFFTD(23S)	62.96±2.83	60.87±2.68	59.03±1.57	64.06±3.36	63.08±3.00	62.52±2.88	64.26±2.14	64.30±2.56	63.46±2.88	61.57±3.07	**64.78±2.93**	63.36±2.74	60.02±3.82	62.19±3.64	61.29±2.10
MFFTD(23T)	62.84±2.84	61.14±2.33	58.64±1.93	63.23±3.24	63.22±3.10	62.39±2.86	64.68±3.25	65.38±2.84	63.68±2.88	63.32±3.16	**65.90±2.87**	63.58±2.69	61.93±3.63	62.59±3.41	61.19±2.38
Average	64.40±1.74	62.42±1.64	60.03±1.40	66.13±2.89	64.83±1.94	65.05±3.00	68.54±4.71	67.92±3.59	65.90±2.69	68.01±2.65	**69.31±2.55**	66.16±2.64	66.07±3.58	67.57±2.96	66.65±2.77

perfectly solves the problem of data imbalance and advanced feature abstraction fusion process in the case of scarce data and improves the fault diagnosis accuracy. By comparing the no temporal dynamic graphs with the complete MFFTD, it was verified that the temporal dynamic graphs could effectively capture the long-term spatial topological dependence between data and increase the learning ability of the model. Compared with the complete MFFTD, no MSRC results show that the multi-scale residual convolutional network realizes a comprehensive capture of the spatial features of flight data and improves the accuracy of the model. In short, these key modules are crucial for the stability and reliability of MFFTD.

4.7. Computational efficiency comparison

In this section, a comparative analysis is conducted on the parameter scale and single-sample fault diagnosis time (fault diagnosis time on July 21st) of MFFTD in comparison with those of the other models, as illustrated in Table 11. This was aimed at demonstrating the differences in computational efficiency among the various models and uncovering the details of their designs and performances. Although MFFTD has a relatively large number of parameters, it demonstrates a reasonable inference time in complex models owing to its excellent network design. Moreover, its accuracy was significantly higher than that of the other models, fully showcasing its advantages in spatio-temporal feature modeling and computational efficiency. Compared with SHNN and ASN, MFFTD exhibits remarkable computational efficiency, and when compared with Cross-Stitch, HDMTL, and NDDR-CNN, MFFTD shows obvious performance superiority. Although its inference time is slightly longer, its high accuracy compensates for this gap. The latter models tend to compromise the complexity of fault diagnosis when attempting to reduce the inference time, whereas MFFTD exhibits balanced accuracy and complexity. In comparison with SVM, MFFTD is slightly slower in terms of the inference speed but is more suitable for high-complexity tasks. SVM, owing to its simple model and fast computation, lacks the ability to model complex spatio-temporal data, and thus, is unable to provide reliable diagnostic results in complex tasks. In conclusion, MFFTD achieves a balance between the parameter scale, inference efficiency, and diagnostic accuracy. It features high accuracy and robustness with a reasonable inference time and performs excellently in complex tasks.

4.8. Feature visualization

In this section, we present conducted extensive visual analysis on the internal feature representation mechanism of MFFTD and conducted subsequent experiments after the convergence of the training module.

4.8.1. Backbone visualization

We visualized the effects of the backbone network before and after training with 90 training samples on the July 21st dataset. As shown in Fig. 11(a), before training, the feature maps of the three convolutional layers are randomly activated and their colors are irregular. Because the parameters of the convolutional kernels have not been optimized and rely only on random weights, it is impossible to extract meaningful features, and the feature-extraction ability is nearly zero. As shown in Fig. 11(b), after training, the features are optimized layer by layer, demonstrating the process of the model learning meaningful features. The features in the Conv1_1 layer are relatively sparse, mainly extracting local low-level features. The Conv2_2 layer begins to present more obvious striped features, with enhanced sensitivity to time series and local patterns. The features in the Conv3_3 layer are more focused, and the highlighted areas are distributed in blocks, indicating that the global high-level feature patterns are captured, which are closely related to the goals of the classification task. Such changes reflect the effectiveness of multi-scale convolutional kernels. MFFTD can capture both local and

Table 10
Results of ablation experiments.

Models	Dataset and Sample Sizes							
	July12 90	July12 120	July13 90	July13 120	July21 90	July21 120	July23 90	July23 120
no MHMIFF-S	67.06±5.78	72.36±6.21	80.27±4.23	82.43±3.97	42.26±6.75	41.95±6.44	39.32±5.96	40.59±6.24
no MHMIFF-T	83.31±4.66	85.23±5.62	84.49±3.79	85.07±3.86	61.72±6.43	68.57±6.04	56.70±5.27	61.09±5.61
no GTDG-S	85.65±3.57	87.26±2.93	85.92±3.43	86.15±2.97	68.96±4.72	73.68±4.31	62.76±4.83	67.81±4.31
no GTDG-T	85.63±3.68	87.27±2.81	85.93±3.31	86.16±3.04	69.52±4.22	72.16±4.09	60.85±4.67	66.52±4.28
no MRCN-S	84.85±1.74	87.85±1.57	85.09±1.97	87.21±1.61	70.47±2.66	75.41±2.38	62.34±3.05	66.10±2.96
no MRCN-T	84.86±1.94	87.86±1.71	85.07±2.03	87.18±1.42	70.67±2.55	75.43±2.18	60.66±3.34	65.73±2.79
MFFTD-S	88.03±1.07	90.38±1.04	89.24±1.56	**90.72±1.23**	73.09±1.94	**77.52±1.63**	64.35±2.78	**69.63±2.46**
MFFTD-T	**88.10±1.32**	**90.69±1.05**	**89.32±1.39**	**90.72±1.44**	**73.33±2.34**	77.46±1.80	**65.38±2.89**	69.52±2.44

Table 11
Parameter and diagnostic time comparisons among different models.

Methods	Parameters	Time consumed (s)	Rankings
SVM	198	0.000020	1
CANN	4.87×10^6	0.001689	4
ASN	5.24×10^4	0.038743	7
SHNN	3.85×10^4	0.039216	8
NDDR-CNN	4.32×10^6	0.001170	2
Cross-Stitch	2.94×10^6	0.001676	3
HDMTL	1.28×10^7	0.002286	5
MFFTD	1.46×10^7	0.003929	6

global features of the data, realize a fine feature representation of the input data, and enhance classification.

4.8.2. Visualization of GTDG

To verify the effectiveness of GTDG, we trained MFFTD with 90 samples from each of the four datasets on July 12th, 13th, 21st, and 23rd under the condition that the initial value of the adjacency matrix was 1. We then visualized the GTDG of the trained model, as shown in Fig. 12. This can be observed from Fig. 12(c) and (d) that the weight distribution of GTDG commences to vary. The specific manifestations are as follows: the associations between certain nodes are fortified, as evidenced by the intensification of the red areas. Simultaneously, the dependencies between some nodes are attenuated, with the emergence of blue areas in tandem. Nevertheless, at this juncture, the weight distribution is relatively dispersed, the global relationship is rather tenuous, and the local dependencies do not exhibit pronounced internode linkages. In Fig. 12(e) and (f), the weight distribution begins to become more concentrated. The red and blue areas are more conspicuous, and their values are lower. These alterations signify that the spatial topological relationships have been thoroughly learned and reinforced by the model, further corroborating that the model is capable of capturing more transitive topological relationships within a global context.

4.8.3. Visualization of MHMIFF

To verify the effectiveness of MHMIFF, we visualized its interior of MHMIFF using the model that was trained with 90 samples on the dataset of July 21st, as shown in Fig. 13. Fig. 13(a) presents the weight distribution of the multi-head attention mechanism after training in the MHMIFF module. The attention mechanism measures the importance of the input features. The higher the weight, the brighter the color, indicating a more significant relationship. In the initial training stage, the heatmap showed a uniform color, indicating that the model treated all features equally without capturing the feature relationships, and there was no significant interaction among the features. As the training progressed, the weight range expanded and the heatmap appeared in a strip shape, with the intensity of the feature interaction increasing and the weights of some features increasing. In the final stage, the heatmap becomes highly sparse and the weight range expands further, demonstrating that the model has clearly identified the most crucial features for

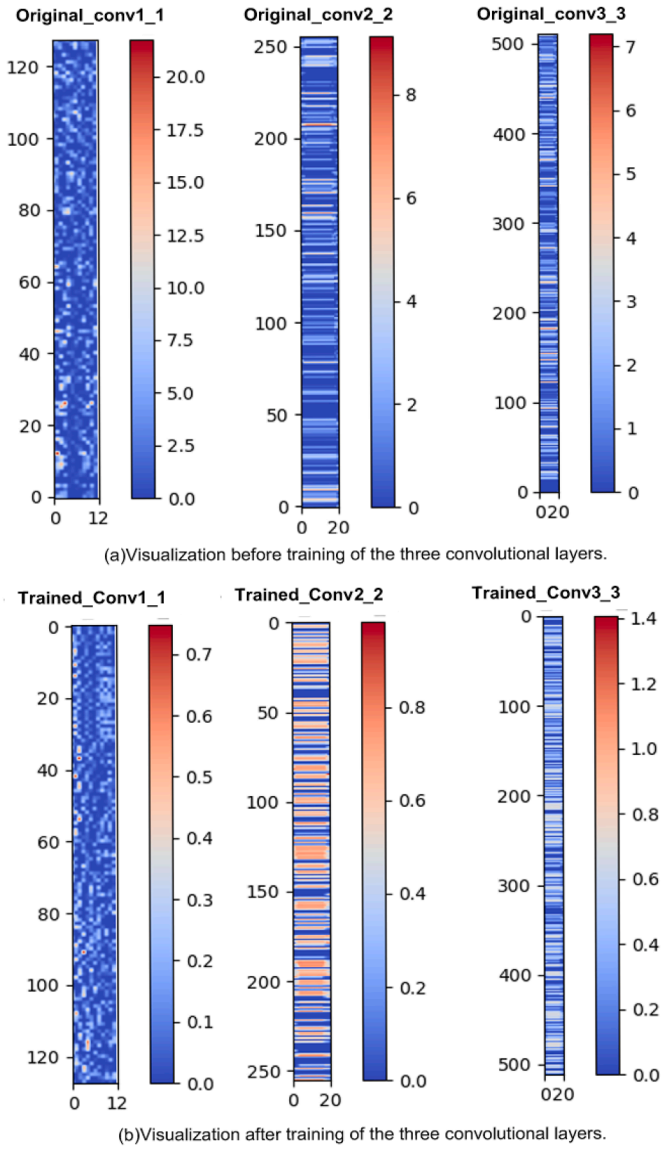


Fig. 11. Visualization of the backbone before and after training.

the task, thus validating the effectiveness of the attention mechanism in feature selection.

Fig. 13(b) presents the interaction patterns of the features output by the encoder of the MHMIFF module. In the initial stage, the heatmap shows a uniform color, indicating a shallow understanding of the feature interactions. The interaction intensities of most features were similar without any hierarchy or sparsity. Subsequently, the highlighted areas in the heatmap suggest that the model begins to capture deep-seated

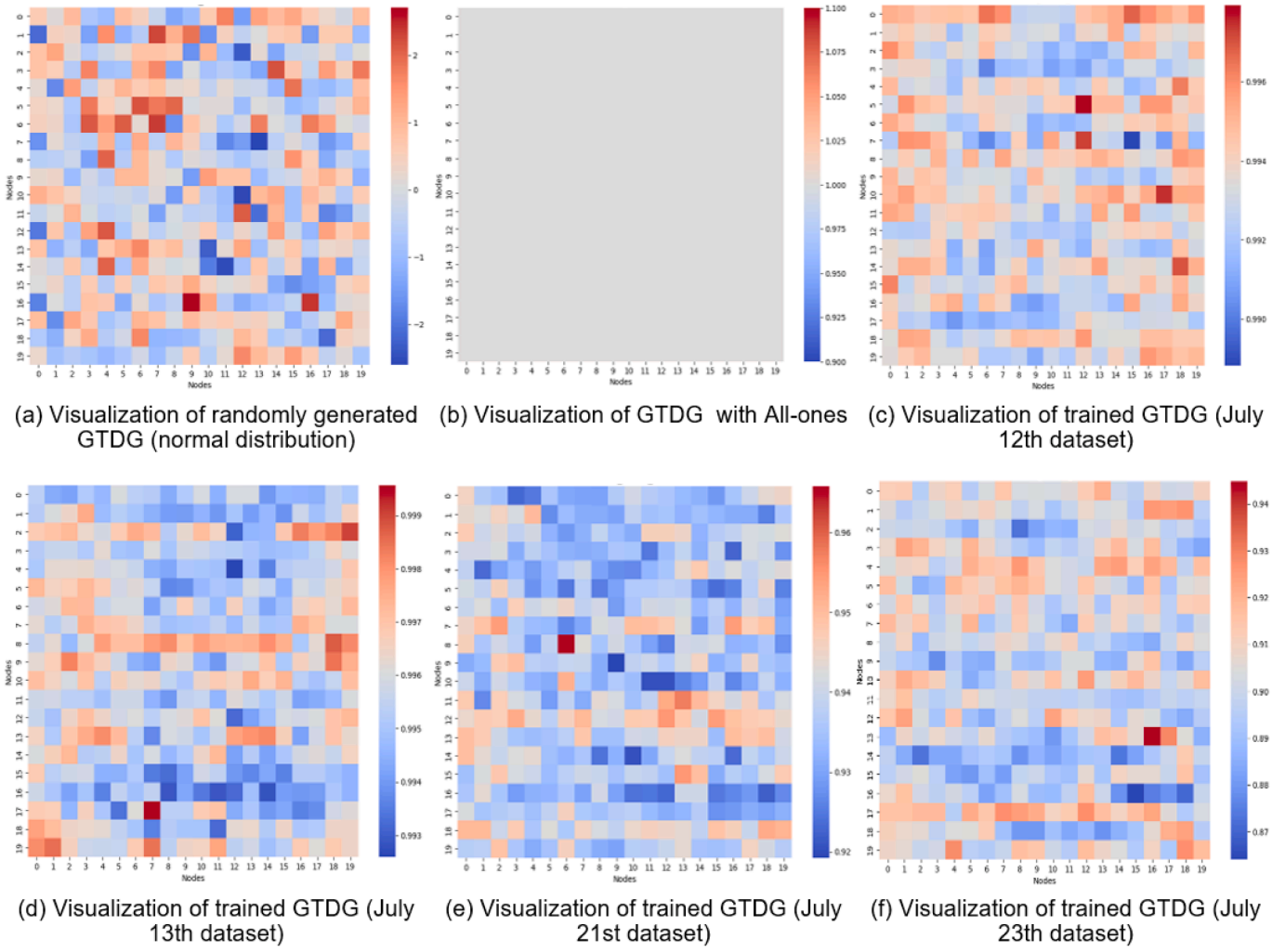


Fig. 12. Visualization of the GTDG before and after training.

relationships and that there are differences in the interaction intensities. Finally, the sparsity increased significantly, with most areas being dark purple, demonstrating that the model can focus on important interactions and ignore irrelevant features. The attention mechanism and encoder are responsible for the feature selection and feature interaction, respectively. They achieved efficient feature fusion and significantly enhanced the performance of the model for downstream tasks.

4.9. Feasibility analysis

In practical engineering, the collection of flight data from FW-UAVs is susceptible to environmental factors such as electromagnetic interference and aerodynamic vibration, which results in the data of the sensors being superimposed with random noise, severely restricting the stability of fault diagnosis models. To verify the engineering applicability of MFFTD in noisy scenarios, this section conducts verification on the model performance under Gaussian white noise interference: Considering the characteristic of scarcity in practical engineering fault data, the test set data injected with Gaussian white noise is tested on the multi-classification model trained using 90 samples on July 21, and 23. The noise generation follows the formula:

$$\mathrm{SNR} = 10\log_{10}\left(\frac{\sigma_x^2}{\sigma_n^2}\right) \tag{29}$$

Where SNR denotes the signal-to-noise ratio (unit: dB), σ_x^2 represents the variance of the original signal, and σ_n^2 represents the variance of the noise signal. The noise intensity covers scenarios from weak to strong noise in gradients ranging from -16 dB to 16 dB with a step of 4 dB, and the classification accuracy is taken as the average of spatial and temporal channels. The experimental results are shown in Fig. 14.

The experimental results show that the accuracy of all models decreases with decreasing SNR (i.e., increased noise). Notably, the MFFTD model significantly outperforms other models across all SNR scenarios and exhibits the smallest drop in accuracy. Furthermore, models trained on the July 23rd dataset demonstrate better stability, as the higher wind speed on that day led to more noise in the collected data compared to the July 21st dataset, enabling more adequate training and thus stronger stability. In conclusion, the experiments fully validate that the unique design of MFFTD endows it with superior noise-resistant robustness and engineering applicability in practical engineering scenarios characterized by scarce fault data and existing noise interference.

Moreover, in practical applications, the parameter scale and single-sample inference time of MFFTD are compatible with common UAV embedded hardware; meanwhile, its sensor integration has simple requirements, only needing a 50Hz sampling frequency and configuration of 12 types of UAV flight attitude sensors. Thus, this technology exhibits favorable practical feasibility in real-world scenarios.

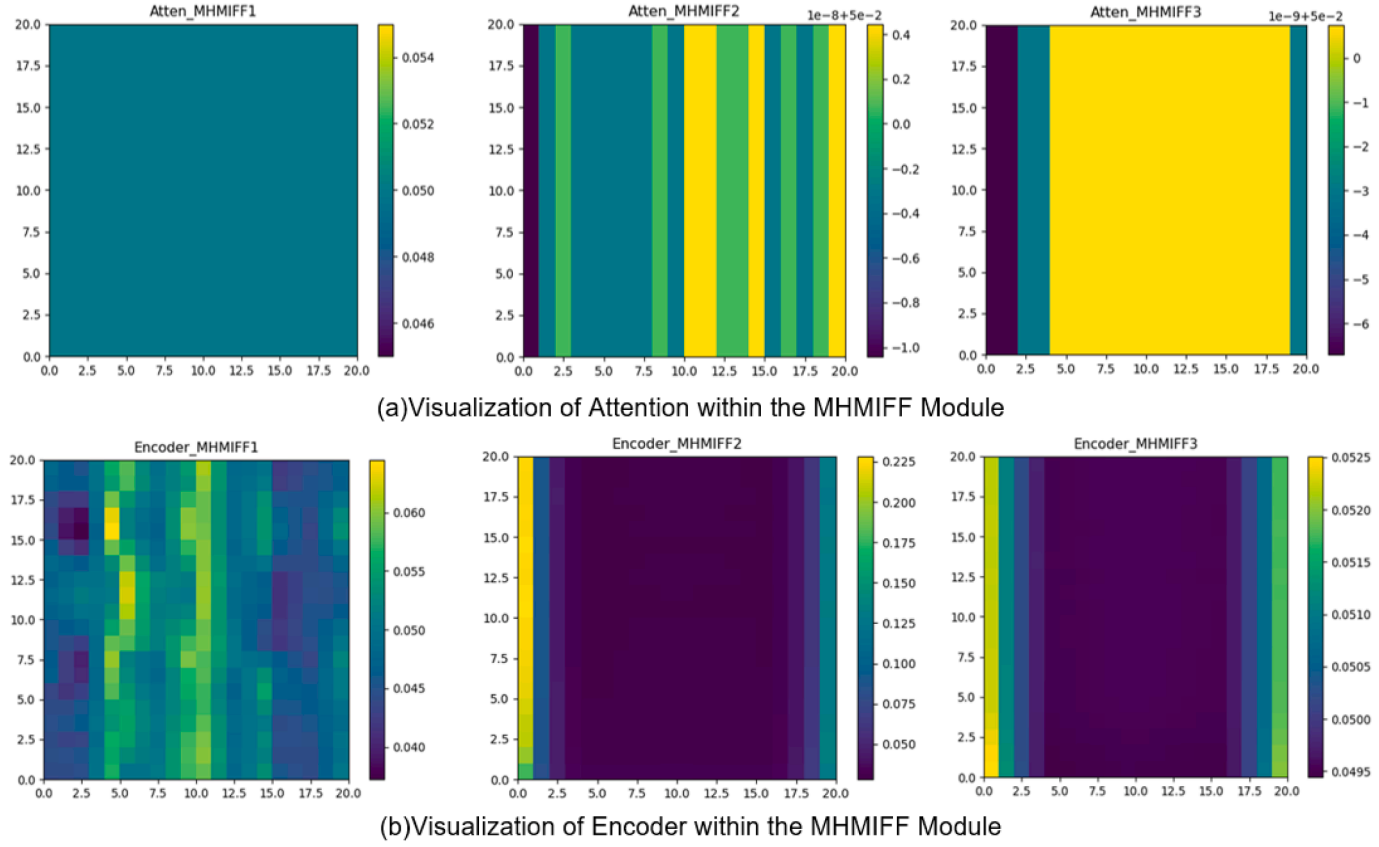
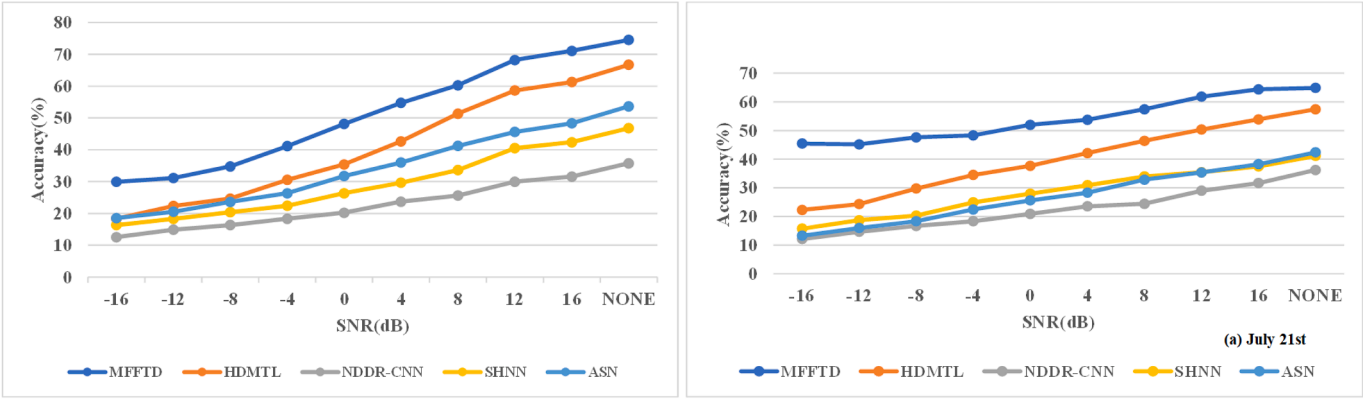


Fig. 13. Visualization of the MHMIFF after training.



(a) Test on July 21 trained model **(b) Test on July 23 trained model**

Fig. 14. Fault diagnosis under different noise environment on trained model.

5. Conclusion

To address the scarcity of flight fault data, this study differs from conventional data augmentation or few-shot learning methods and focuses on the spatiotemporal correlation characteristics and multidimensional heterogeneity of UAV flight data. Based on small samples, we proposed MFFTD. We designed a multi-scale residual convolutional network to comprehensively capture the spatial features of flight data, introduced a global temporal dynamic graph to achieve topology enhancement, and utilized an LSTM network to mine the temporal features deeply. In addition, a novel multi-head mutual information self-attention feature fusion module was designed to dynamically weigh the spatiotemporal features, addressing the issue of feature fusion imbalance. Under the conditions of limited flight data, the fault diagnosis accuracy of MFFTD significantly surpasses that of the previous models. However, the model's multi-component architecture introduces high complexity and a large parameter count, limiting its deployment in resource-constrained UAV systems; meanwhile, practical deployment must contend with complex environmental disturbances like temperature fluctuations, electromagnetic interference, and sensor noise. Future research should thus focus on simplifying the structure to reduce computational costs without compromising accuracy, while further exploring robustness under such varying operating conditions. Furthermore, MFFTD has been experimentally verified to exhibit stability in the generalization dimensions of cross-time-series, cross-scenario, and cross-distribution features, laying a foundation for its extended applications. Meanwhile, it can be further extended to potential generalization directions, including cross-equipment

models, cross-workloads, and cross-environmental interferences, incorporating the fault diagnosis needs of other UAVs or multimodal flight data, and integrating with advanced technologies to enhance robustness and real-time performance, to better meet the needs of UAV safe flight missions.

CRediT authorship contribution statement

Guanjun Li: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Haoyu Gui:** Writing – review & editing, Validation, Software, Methodology, Investigation, Data curation; **Jianguang Lu:** Writing – review & editing, Supervision, Funding acquisition; **Xianghong Tang:** Writing – review & editing, Conceptualization; **Xiaoyu Gao:** Writing – review & editing.

Funding

This work was supported by Guizhou Provincial Basic Research Program (Natural Science), China (grant QKHJC-ZK(2024)YB062), Guizhou Provincial Key Technology R&D Program, China (grant QKHZC(2025)YB146) and Guizhou University Basic Research Program, China (grant GDJC(2024)09).

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank the reviewers for their insightful comments and helpful suggestions.

References

[1] H. Wang, H. Cheng, H. Hao, The use of unmanned aerial vehicle in military operations, in: International Conference on Man-Machine-Environment System Engineering, Springer, 2020, pp. 939–945.
[2] R. Sharma, R. Arya, UAV based long range environment monitoring system with industry 5.0 perspectives for smart city infrastructure, Comput. Ind. Eng. 168 (2022) 108066.
[3] P. Radoglou-Grammatikis, P. Sarigiannidis, T. Lagkas, I. Moscholios, A compilation of UAV applications for precision agriculture, Comput. Netw. 172 (2020) 107148.
[4] F. Outay, H.A. Mengash, M. Adnan, Applications of unmanned aerial vehicle (UAV) in road safety, traffic and highway infrastructure management: recent advances and challenges, Transp. Res. Part A Policy Pract. 141 (2020) 116–129.
[5] M. Jones, S. Djahel, K. Welsh, Path-planning for unmanned aerial vehicles with environment complexity considerations: a survey, ACM Comput. Surv. 55 (11) (2023) 1–39.
[6] R. Telford, S. Galloway, Fault classification and diagnostic system for unmanned aerial vehicle electrical networks based on hidden Markov models, IET Electr. Syst. Transp. 5 (3) (2015) 103–111.
[7] F.R. López-Estrada, J.-C. Ponsart, D. Theilliol, Y. Zhang, C.-M. Astorga-Zaragoza, LPV model-based tracking control and robust sensor fault diagnosis for a quadrotor UAV, J. Intell. Rob. Syst. 84 (2016) 163–177.
[8] F. Chen, R. Jiang, K. Zhang, B. Jiang, G. Tao, Robust backstepping sliding-mode control and observer-based fault estimation for a quadrotor UAV, IEEE Trans. Ind. Electron. 63 (8) (2016) 5044–5056.
[9] A. Hasan, T.A. Johansen, Model-based actuator fault diagnosis in multirotor UAVs, in: 2018 International Conference on Unmanned Aircraft Systems (ICUAS), IEEE, 2018, pp. 1017–1024.
[10] R. Wang, C. Zhao, Y. Bai, W. Du, J. Wang, An actuator fault detection and reconstruction scheme for hex-rotor unmanned aerial vehicle, IEEE Access 7 (2019) 93937–93951.
[11] J. Gao, Q. Zhang, J. Chen, EKF-based actuator fault detection and diagnosis method for tilt-rotor unmanned aerial vehicles, Math. Prob. Eng. 2020 (1) (2020) 8019017.
[12] M. Bronz, E. Baskaya, D. Delahaye, S. Puechmore, Real-time fault detection on small fixed-wing UAVs using machine learning, in: 2020 AIAA/IEEE 39th Digital Avionics Systems Conference (DASC), IEEE, 2020, pp. 1–10.
[13] A. Altinors, F. Yol, O. Yaman, A sound based method for fault detection with statistical feature extraction in UAV motors, Appl. Acoustics 183 (2021) 108325.
[14] K. He, D. Yu, D. Wang, M. Chai, S. Lei, C. Zhou, Graph attention network-based fault detection for UAVs with multivariant time series flight data, IEEE Trans. Instrum. Meas. 71 (2022) 1–13.
[15] S. Wang, Z. Jia, Z. Liu, Y. Tang, X. Qin, X. Wang, Self-supervised contrast learning based UAV fault detection and interpretation with spatial–temporal information of multivariate flight data, Expert Syst. Appl. 267 (2024) 126156.
[16] C. Li, S. Li, A. Zhang, L. Yang, E. Zio, M. Pecht, K. Gryllias, A Siamese hybrid neural network framework for few-shot fault diagnosis of fixed-wing unmanned aerial vehicles, J. Comput. Des. Eng. 9 (4) (2022) 1511–1524.
[17] Y. Zhang, S. Li, A. Zhang, X. An, FW-UAV fault diagnosis based on knowledge complementary network under small sample, Mech. Syst. Signal Process. 215 (2024) 111418.
[18] Y. Zhang, S. Li, L. Yang, Y. Zhu, Z. Liao, Y. Wang, C. Li, H. Wang, FW-UAV fault diagnosis based on multilevel task knowledge supplement network under small samples, IEEE Trans. Instrum. Meas. 73 (2024) 1–14.
[19] K. Guo, N. Wang, D. Liu, X. Peng, Uncertainty-aware LSTM based dynamic flight fault detection for UAV actuator, IEEE Trans. Instrum. Meas. 72 (2022) 1–13.
[20] X. Chu, X. Zhou, Q. Bu, Q. Miao, Sensor fault detection for UAVs using improved self-attention LSTM network with similarity space mapping, IEEE Trans. Instrum. Meas. 73 (2024) 1–12.
[21] N. Ceylan, E. Sönmez, S. Kaçar, Cost effective detection of uneven mounting fault in rotary wing drone motors with a CNN based method, Signal Image Video Process. 18 (11) (2024) 8049–8059.
[22] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324.
[23] S. Yin, Z. Xiang, A hyper-heuristic algorithm via proximal policy optimization for multi-objective truss problems, Expert Syst. Appl. 256 (2024) 124929.
[24] D. Guo, M. Zhong, H. Ji, Y. Liu, R. Yang, A hybrid feature model and deep learning based fault diagnosis for unmanned aerial vehicle sensors, Neurocomputing 319 (2018) 155–163.
[25] T. Gao, W. Sheng, M. Zhou, B. Fang, L. Zheng, MEMS inertial sensor fault diagnosis using a cnn-based data-driven method, Int. J. Pattern Recognit. Artif. Intell. 34 (14) (2020) 2059048.
[26] C. Du, X. Zhang, R. Zhong, F. Li, F. Yu, Y. Rong, Y. Gong, Unmanned aerial vehicle rotor fault diagnosis based on interval sampling reconstruction of vibration signals and a one-dimensional convolutional neural network deep learning method, Meas. Sci. Technol. 33 (6) (2022) 065003.
[27] P. Yang, W. Li, C. Wen, P. Liu, Fault diagnosis method of multi-rotor UAV based on one-dimensional convolutional neural network with adaptive batch normalization algorithm, Meas. Sci. Technol. 35 (2) (2023) 025102.
[28] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 1–9.
[29] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[30] J. Yao, W. Li, Z. Zheng, X. Jia, CLGNN: UAV fault diagnosis via causal learning and graph neural network, in: 2024 International Joint Conference on Neural Networks (IJCNN), IEEE, 2024, pp. 1–8.
[31] C.D.T. Barros, M.R.F. Mendonça, A.B. Vieira, A. Ziviani, A survey on embedding dynamic graphs, ACM Comput. Surv. (CSUR) 55 (1) (2021) 1–37.
[32] Y. Yuan, X. Liang, X. Wang, D.-Y. Yeung, A. Gupta, Temporal dynamic graph LSTM for action-driven video object detection, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 1801–1810.
[33] H. Liu, D. Yang, X. Liu, X. Chen, Z. Liang, H. Wang, Y. Cui, J. Gu, TodyNet: temporal dynamic graph neural network for multivariate time series classification, Inf. Sci. 677 (2024) 120914.
[34] H. Gui, G. Li, X. Tang, J. Lu, CATodyNet: cross-attention temporal dynamic graph neural network for multivariate time series classification, Knowl. Based Syst. 300 (2024) 112210.
[35] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L.u. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, 2017, pp. 6000–6010.
[36] P. Xiong, Z. Li, Y. Li, S. Huang, C. Liu, F. Gu, Fault diagnosis of UAV based on adaptive Siamese network with limited data, IEEE Trans. Instrum. Meas. 72 (2023) 1–11.
[37] S. Wang, Z. Liu, Z. Jia, Y. Tang, G. Zhi, X. Wang, Fault detection for UAVs with spatial-temporal learning on multivariate flight data, IEEE Trans. Instrum. Meas. 73 (2024) 1–17.
[38] Y. Zhang, S. Li, Y. Gu, Q. He, P. Zhou, A. Zhang, UAV fault diagnosis based on collaborative sharing of generic and task-oriented features, Expert Syst. Appl. 296 (2025) 128940.
[39] H. Peng, Z. Mao, B. Jiang, Y. Cheng, Multiscale spatial-temporal Bayesian graph conv-transformer-based distributed fault diagnosis for UAVs swarm system, IEEE Trans. Aerosp. Electron. Syst. 60 (5) (2024) 6894–6909.
[40] S. Targ, D. Almeida, K. Lyman, ResNet in ResNet: generalizing residual architectures, (2016). arXiv preprint arXiv:1603.08029.
[41] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[42] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.
[43] M.D. Tandale, J. Valasek, Fault-tolerant structured adaptive model inversion control, J. Guid. Contr. Dyn. 29 (3) (2006) 635–642.

[44] M.-Q. Tran, M.-K. Liu, Q.-V. Tran, T.-K. Nguyen, Effective fault diagnosis based on wavelet and convolutional attention neural network for induction motors, IEEE Trans. Instrum. Meas. 71 (2021) 1–13.
[45] Y. Gao, J. Ma, M. Zhao, W. Liu, A.L. Yuille, NDDR-CNN: layerwise feature fusing in multi-task cnns by neural discriminative dimensionality reduction, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 3205–3214.
[46] I. Misra, A. Shrivastava, A. Gupta, M. Hebert, Cross-stitch networks for multi-task learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 3994–4003.